

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	Lakshmi Sai Ram K
ROLL NO	AV.SC.U4CSE25213
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasad
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-13
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	14-19
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	20-25
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	26-29
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	30-38

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10)Search data in SLL 11)Display or Traverse 12)Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10)Exit	39-49
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	49-59
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	60-79
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion	80-91

	3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	92-99
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	100-104
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	105-117

WEEK – 1

PROGRAM-1(A):

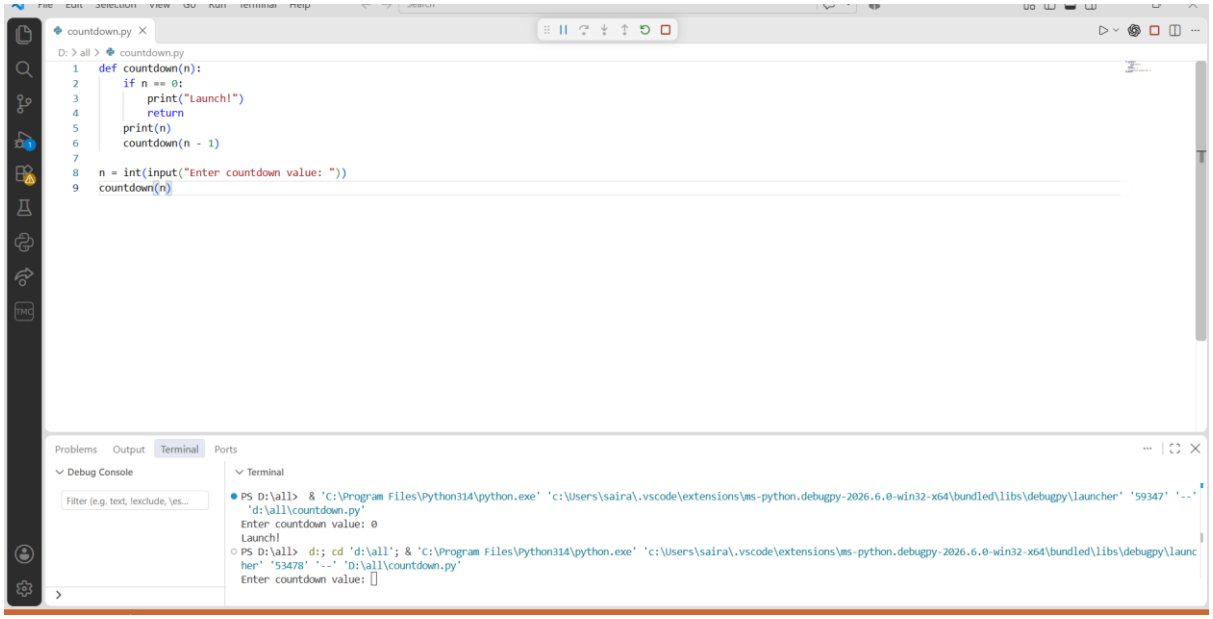
AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by “Launch!”.

CODE:

```
def countdown(n):
    if n == 0:
        print("Launch!")
        return
    print(n)
    countdown(n - 1)

n = int(input("Enter countdown value: "))
countdown(n)
```

OUTPUT:


countdown.py
D: > all > countdown.py
def countdown(n):
if n == 0:
print("Launch!")
return
print(n)
countdown(n - 1)
n = int(input("Enter countdown value: "))
countdown(n)
Problems
Output
Terminal
Ports
Debug Console
Filter (e.g. text, !exclude, \es...
Terminal
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59347' '--' 'd:\all\countdown.py'
Enter countdown value: 0
Launch!
PS D:\all> d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53478' '--' 'D:\all\countdown.py'
Enter countdown value:

PROGRAM-1(B):

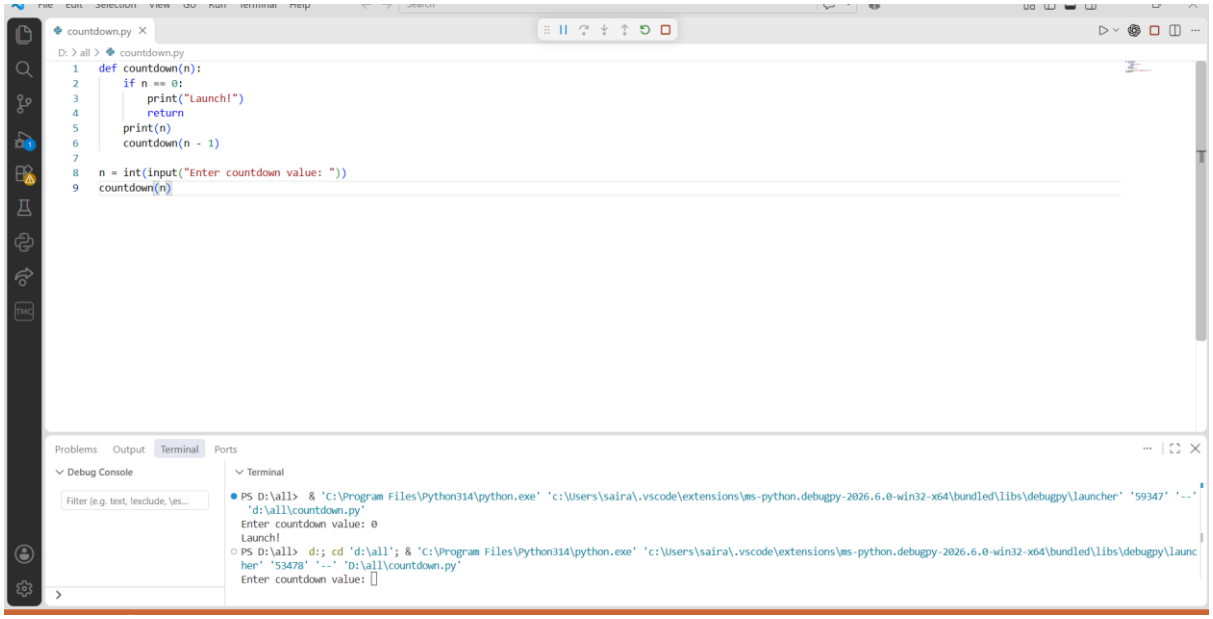
AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years.

CODE:

```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

p = float(input('get me the amount:\t'))
n = int(input('agreed years:\t'))
print(power(p, n))
```

OUTPUT:


countdown.py
D: > all > countdown.py
def countdown(n):
if n == 0:
print("Launch!")
return
print(n)
countdown(n - 1)
n = int(input("Enter countdown value: "))
countdown(n)
Problems
Output
Terminal
Ports
Debug Console
Filter (e.g. text, !exclude, \es...
Terminal
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59347' '--' 'd:\all\countdown.py'
Enter countdown value: 0
Launch!
PS D:\all> d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53478' '--' 'D:\all\countdown.py'
Enter countdown value:

PROGRAM-1(C):

AIM: A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

```
def recursive_search(arr, key, index):
    if index == len(arr):
        return -1
    if arr[index] == key:
        return index
    return recursive_search(arr, key, index + 1)

arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter employee ID to search: "))

result = recursive_search(arr, key, 0)

if result == -1:
    print("Employee ID not found")
else:
    print("Employee ID found at index", result)
```

OUTPUT:


recursivesearch.py
D: > all > recursivesearch.py > ...
def recursive_search(arr, key, index):
if index == len(arr):
return -1
if arr[index] == key:
return index
return recursive_search(arr, key, index + 1)
arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter employee ID to search: "))
result = recursive_search(arr, key, 0)
if result == -1:
print("Employee ID not found")
else:
print("Employee ID found at index", result)
Problems
Output
Terminal
Ports
Debug Console
Filter (e.g. text, !exclude, \es...
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58614' '--'
'd:\all\recursivesearch.py'
Enter elements: 101 102 103 104 105 106 107
Enter employee ID to search: 102
Employee ID found at index 1
PS D:\all>

PROGRAM-1(D):

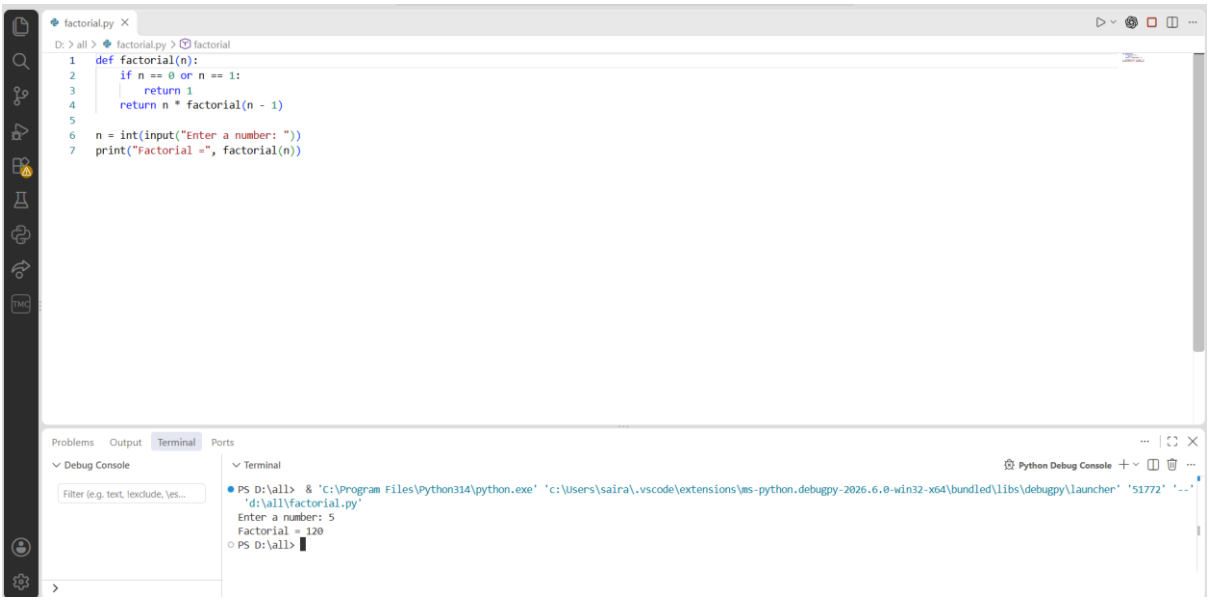
AIM: In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

n = int(input("Enter a number: "))
print("Factorial =", factorial(n))
```

OUTPUT:



PROGRAM-1(E):

AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

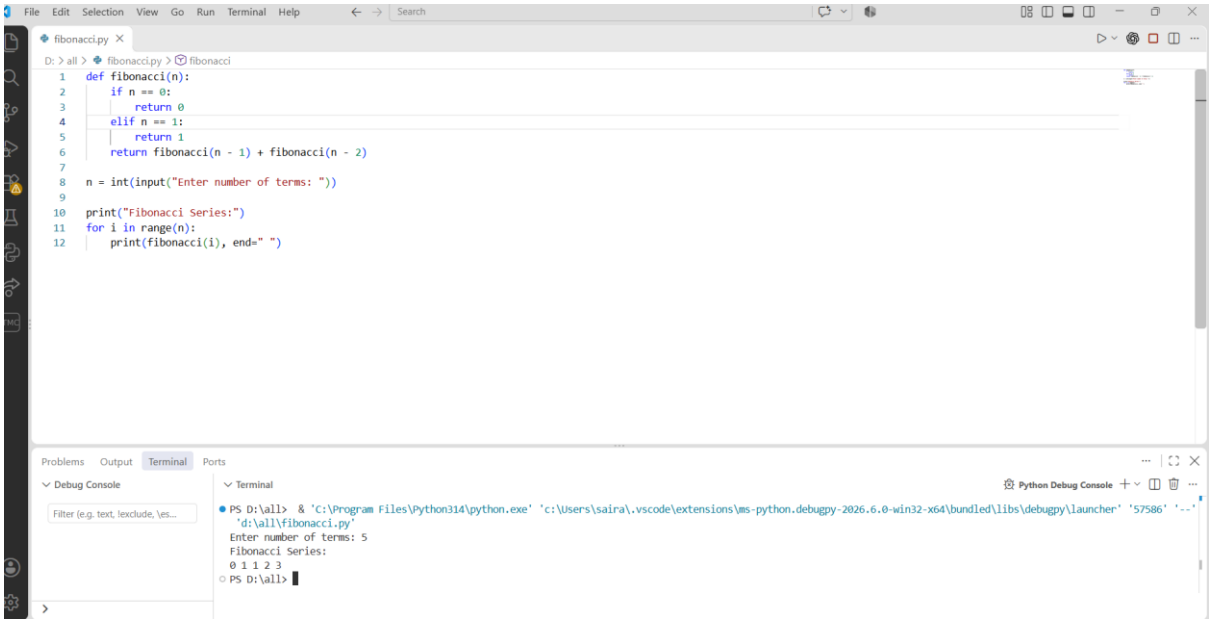
CODE:

```
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number of terms: "))

print("Fibonacci Series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

OUTPUT:


File Edit Selection View Go Run Terminal Help
Search
fibonacci.py
D: > all > fibonacci.py > fibonacci
def fibonacci(n):
if n == 0:
return 0
elif n == 1:
return 1
return fibonacci(n - 1) + fibonacci(n - 2)
n = int(input("Enter number of terms: "))
print("Fibonacci Series:")
for i in range(n):
print(fibonacci(i), end=" ")
Problems Output Terminal Ports
Debug Console
Filter (e.g. text, !exclude, \es...
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '57586' '--'
'd:\all\fibonacci.py'
Enter number of terms: 5
Fibonacci Series:
0 1 1 2 3
PS D:\all>

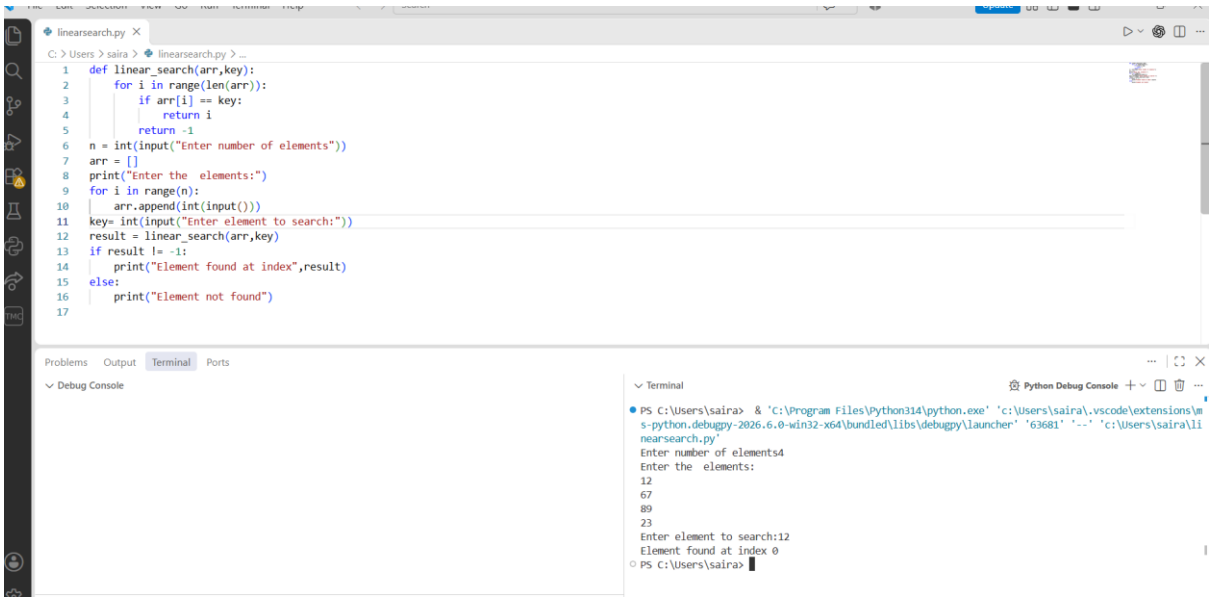
PROGRAM-2(A):

AIM: Write a python program to implement linear search.

CODE:

```
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
        return -1
n = int(input("Enter number of elements"))
arr = []
print("Enter the  elements")
for i in range(n):
    arr.append(int(input()))
key= int(input("Enter element to search"))
result = linear_search(arr,key)
if result != -1:
    print("Element found at index",result)
else:
    print("Element not found")
```

OUTPUT:


linearsearch.py
C: > Users > saira > linearsearch.py > ...
def linear_search(arr,key):
for i in range(len(arr)):
if arr[i] == key:
return i
return -1
n = int(input("Enter number of elements"))
arr = []
print("Enter the elements:")
for i in range(n):
arr.append(int(input()))
key= int(input("Enter element to search:"))
result = linear_search(arr,key)
if result != -1:
print("Element found at index",result)
else:
print("Element not found")
Problems
Output
Terminal
Ports
Debug Console
Terminal
Python Debug Console
PS C:\Users\saira> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63681' '--' 'c:\Users\saira\linearsearch.py'
Enter number of elements4
Enter the elements:
12
67
89
23
Enter element to search:12
Element found at index 0
PS C:\Users\saira>

PROGRAM-2(B):

AIM: Write a python program to implement binary search with and without sorted array as an input.

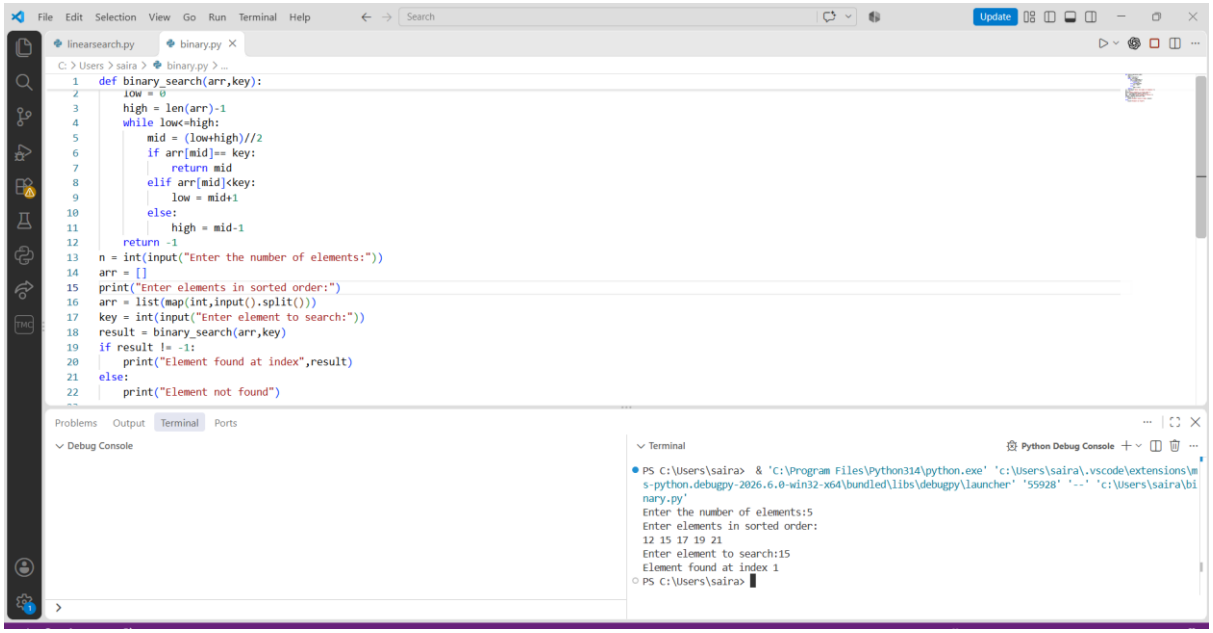
With sorted Array

CODE:

```
def binary_search(arr,key):
    low = 0
    high = len(arr)-1
    while low<=high:
        mid = (low+high)//2
        if arr[mid]== key:
            return mid
        elif arr[mid]<key:
            low = mid+1
        else:
            high = mid-1
    return -1
n = int(input("Enter the number of elements:"))
arr = []
print("Enter elements in sorted order:")
arr = list(map(int,input().split()))
key = int(input("Enter element to search:"))
result = binary_search(arr,key)
if result != -1:
```

```
    print("Element found at index",result)
else:
    print("Element not found")
```

OUTPUT:



With unsorted array

Code:

```
def binary_search(arr,key):
    low = 0
    high = len(arr)-1
    while low<=high:
        mid = (low+high)//2
        if arr[mid]== key:
            return mid
        elif arr[mid]<key:
            low = mid+1
        else:
            high = mid-1
    return -1
arr = list(map(int, input("Enter the elements: ").split()))

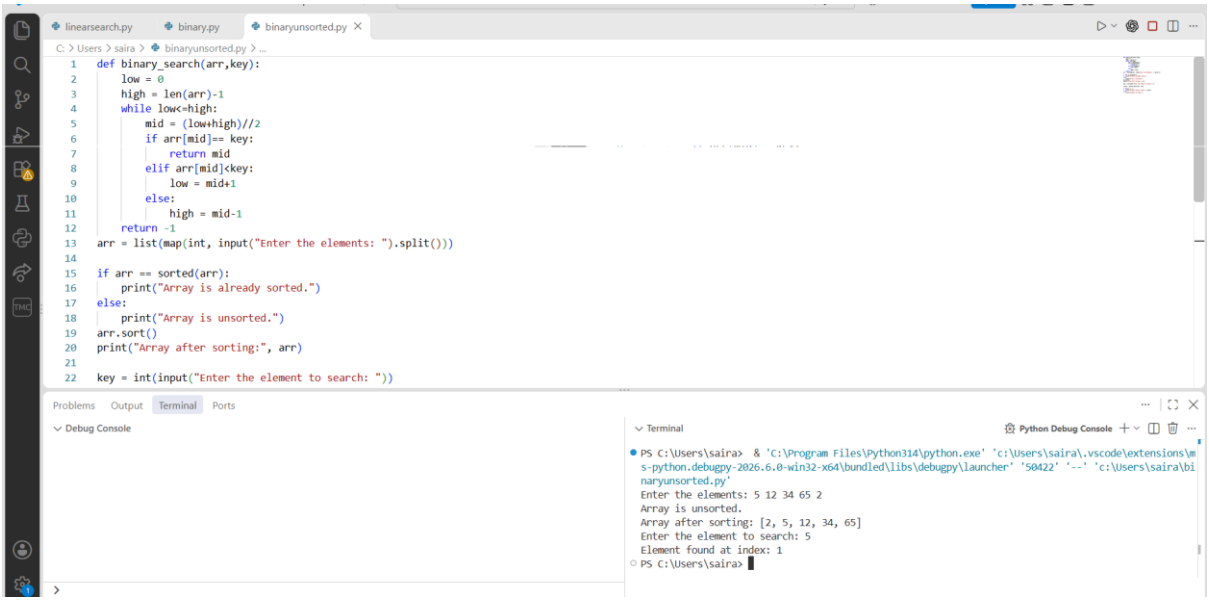
if arr == sorted(arr):
    print("Array is already sorted.")
else:
    print("Array is unsorted.")
arr.sort()
print("Array after sorting:", arr)

key = int(input("Enter the element to search: "))

result = binary_search(arr, key)
```

```
if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found.")
```

OUTPUT:



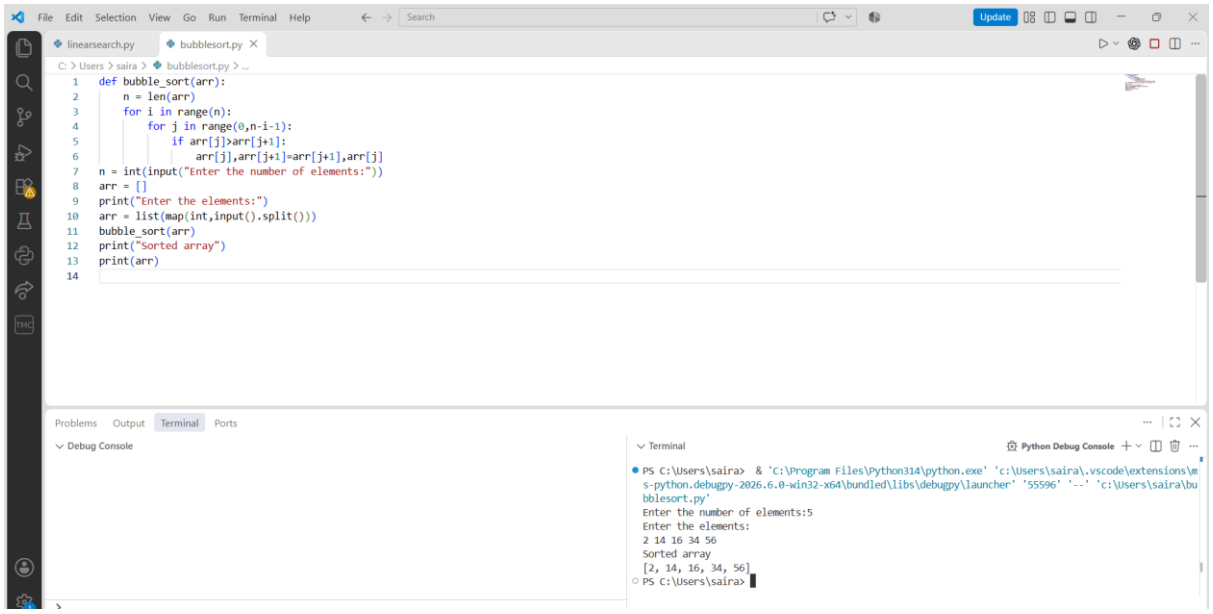
PROGRAM-3(A):

AIM: Write a python program to implement bubble sort.

CODE:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0,n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
arr = list(map(int,input().split()))
bubble_sort(arr)
print("Sorted array")
print(arr)
```

OUTPUT:


File Edit Selection View Go Run Terminal Help
Search
Update
linearsearch.py
bubblesort.py
C: > Users > saira > bubblesort.py > ...
def bubble_sort(arr):
n = len(arr)
for i in range(n):
for j in range(0,n-i-1):
if arr[j]>arr[j+1]:
arr[j],arr[j+1]=arr[j+1],arr[j]
n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
arr = list(map(int,input().split()))
bubble_sort(arr)
print("Sorted array")
print(arr)
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS C:\Users\saira> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55596' '--' 'c:\Users\saira\bubblesort.py'
Enter the number of elements:5
Enter the elements:
2 14 16 34 56
Sorted array
[2, 14, 16, 34, 56]
PS C:\Users\saira>

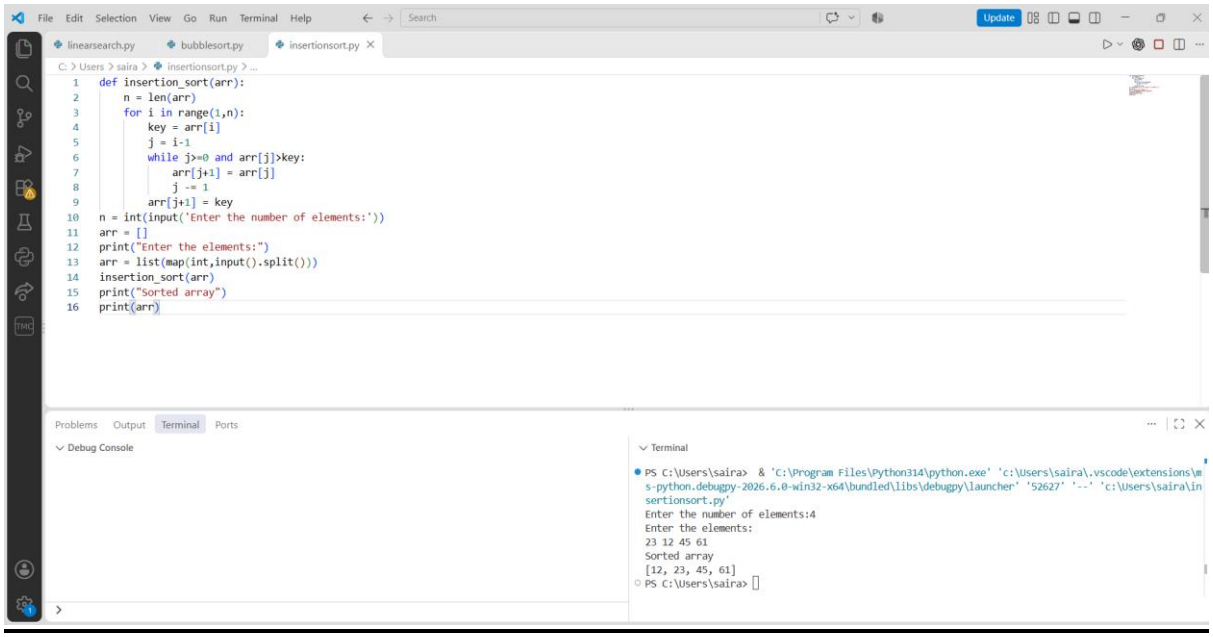
PROGRAM-3(B):

AIM: Write a python program to implement insertion sort.

CODE:

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1,n):
        key = arr[i]
        j = i-1
        while j>=0 and arr[j]>key:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
n = int(input('Enter the number of elements:'))
arr = []
print("Enter the elements:")
arr = list(map(int,input().split()))
insertion_sort(arr)
print("Sorted array")
print(arr)
```

OUTPUT:


File Edit Selection View Go Run Terminal Help
Search
Update
linearsearch.py
bubblesort.py
insertionsort.py
C: > Users > saira > insertionsort.py > ...
def insertion_sort(arr):
n = len(arr)
for i in range(1,n):
key = arr[i]
j = i-1
while j>=0 and arr[j]>key:
arr[j+1] = arr[j]
j -= 1
arr[j+1] = key
n = int(input('Enter the number of elements:'))
arr = []
print("Enter the elements:")
arr = list(map(int,input().split()))
insertion_sort(arr)
print("Sorted array")
print(arr)
Problems Output Terminal Ports
Debug Console
Terminal
PS C:\Users\saira> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52627' '--' 'c:\Users\saira\insertionsort.py'
Enter the number of elements:4
Enter the elements:
23 12 45 61
Sorted array
[12, 23, 45, 61]
PS C:\Users\saira>

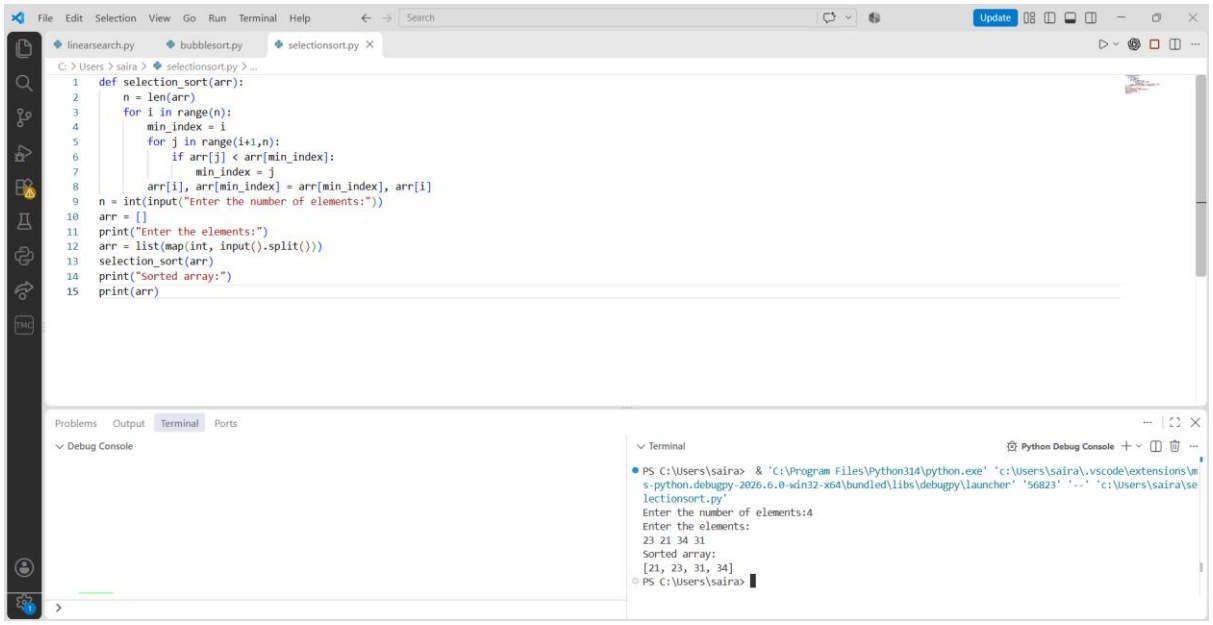
PROGRAM-3(C):

AIM: Write a python program to implement selection sort.

CODE:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i+1,n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
arr = list(map(int, input().split()))
selection_sort(arr)
print("Sorted array:")
print(arr)
```

OUTPUT:


File Edit Selection View Go Run Terminal Help
Search
Update
linearsearch.py
bubblesort.py
selectionsort.py
C: > Users > saira > selectionsort.py > ...
def selection_sort(arr):
n = len(arr)
for i in range(n):
min_index = i
for j in range(i+1,n):
if arr[j] < arr[min_index]:
min_index = j
arr[i], arr[min_index] = arr[min_index], arr[i]
n = int(input("Enter the number of elements:"))
arr = []
print("Enter the elements:")
arr = list(map(int, input().split()))
selection_sort(arr)
print("Sorted array:")
print(arr)
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS C:\Users\saira> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56823' '--' 'c:\Users\saira\selectionsort.py'
Enter the number of elements:4
Enter the elements:
23 21 34 31
Sorted array:
[21, 23, 31, 34]
PS C:\Users\saira>

PROGRAM-4(A):

AIM: Write a python program to implement quick sort.

CODE:

```
def partition(arr, low, high):
    pivot = arr[low]
    i = low + 1
    j = high
    while True:
        while i <= high and arr[i] <= pivot:
            i += 1
        while j >= low and arr[j] > pivot:
            j -= 1
        if i >= j:
            break
        arr[i], arr[j] = arr[j], arr[i]
    arr[low], arr[j] = arr[j], arr[low]
    return j

def quick_sort(arr, low, high):
    if low < high:
        pivot_index = partition(arr, low, high)
        quick_sort(arr, low, pivot_index - 1)
        quick_sort(arr, pivot_index + 1, high)

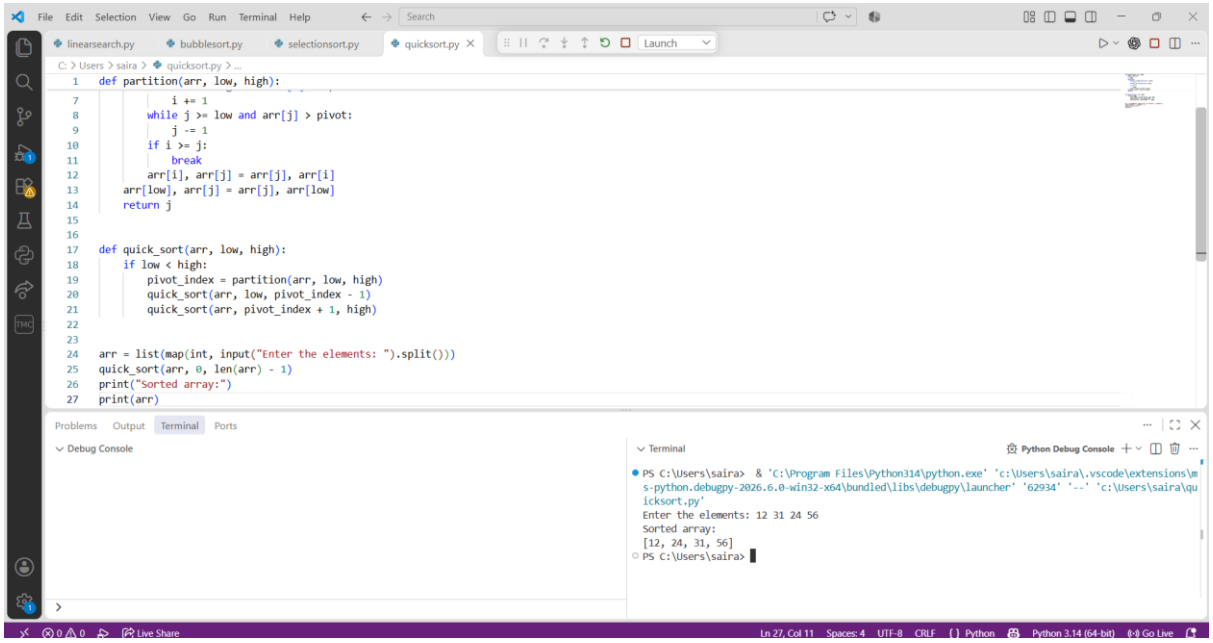
arr = list(map(int, input("Enter the elements: ").split()))
```

```
quick_sort(arr, 0, len(arr) - 1)

print("Sorted array:")

print(arr)
```

OUTPUT:



PROGRAM-4(B):

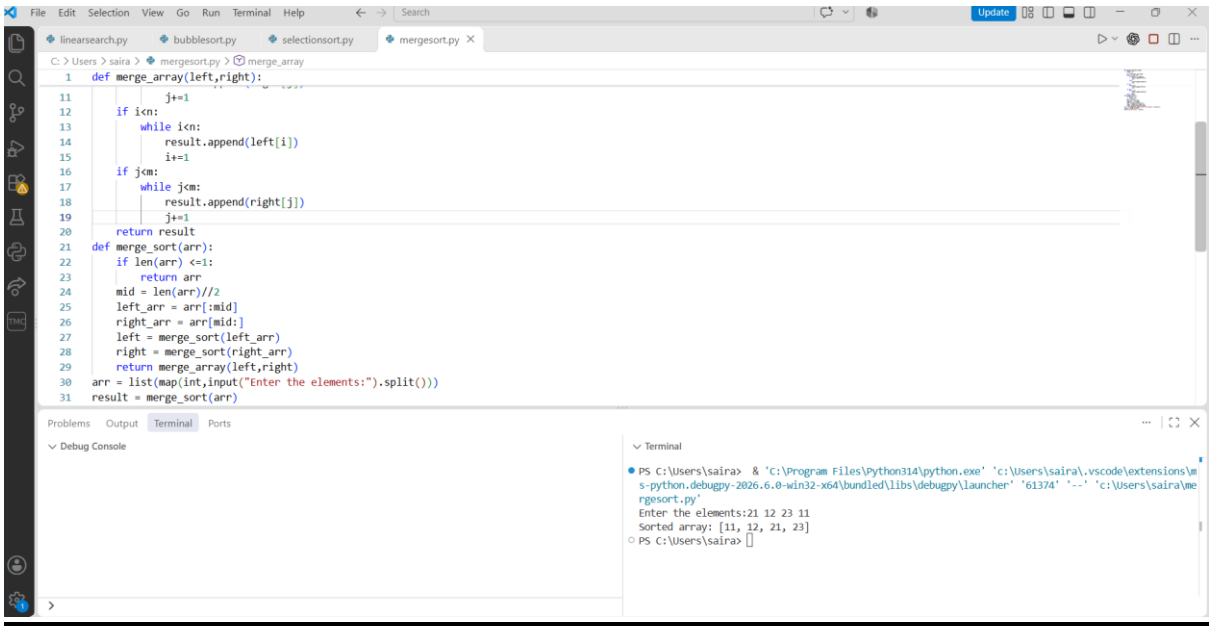
AIM: Write a python program to implement merge sort.

CODE:

```
def merge_array(left,right):
    result = []
    i = j = 0
    n,m = len(left),len(right)
    while i<n and j<m:
        if left[i] <= right[j]:
            result.append(left[i])
            i+=1
        else:
            result.append(right[j])
            j+=1
    if i<n:
        while i<n:
            result.append(left[i])
            i+=1
    if j<m:
        while j<m:
            result.append(right[j])
            j+=1
    return result
def merge_sort(arr):
    if len(arr) <=1:
```

```
        return arr
    mid = len(arr)//2
    left_arr = arr[:mid]
    right_arr = arr[mid:]
    left = merge_sort(left_arr)
    right = merge_sort(right_arr)
    return merge_array(left,right)
arr = list(map(int,input("Enter the elements:").split()))
result = merge_sort(arr)
print("Sorted array:", result)
```

OUTPUT:



PROGRAM-5(A):

AIM: Write a python program to implement a Singly Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Sort the data of SLL
10) Search data in SLL
11) Display or Traverse
12) Exit

CODE:

```
class Node:
    def __init__(self,data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    def create(self):
        n = int(input("Enter number of nodes"))
        for i in range(n):
            data = int(input("Enter data"))
            self.insert_end(data)
```

```
def insert_begin(self,data):
    new =Node(data)
    new.next = self.head
    self.head = new
def insert_end(self,data):
    new = Node(data)
    if self.head is None:
        self.head = new
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new
def insert_index(self,data,index):
    if index < 0:
        print("Invalid Index")
        return
    if index == 0:
        self.insert_begin(data)
        return
    temp = self.head
    for i in range(index-1):
        if temp is None:
            print("Invalid index")
            return
        temp = temp.next
```

```
        if temp is None:
            print("Invalid index")
            return
        new = Node(data)
        new.next = temp.next
        temp.next = new
    def delete_begin(self):
        if self.head is None:
            print("there is no node to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted value=",temp.data)
    def delete_end(self):
        if self.head is None:
            print("there is no node to delete")
        elif self.head.next is None:
            print("Deleted value=",self.head.data)
            self.head = None
        else:
            temp = self.head
            while temp.next.next:
                temp = temp.next
            print("Deleted value=",temp.next.data)
            temp.next = None
    def delete_value(self,value):
```

```
        if self.head is None:
            print("there is no node to delete")
            return
        if self.head.data == value:
            print("Deleted value=",self.head.data)
            self.head = self.head.next
            return
        temp = self.head
        while temp.next and temp.next.data != value:
            temp = temp.next
        if temp.next is None:
            print("Value not found")
            return
        else:
            print("Deleted value=",temp.next.data)
            temp.next = temp.next.next
    def display(self):
        if self.head is None:
            print("List is empty")
        else:
            temp = self.head
            while temp:
                print(temp.data,end=" ")
                temp = temp.next
            print()
    def count(self):
```

```
        temp = self.head
        count = 0
        while temp:
            count += 1
            temp = temp.next
        print("Number of nodes in the list:",count)
    def search(self,value):
        temp = self.head
        index = 0
        while temp:
            if temp.data == value:
                print("Value found at index",index)
                return
            temp = temp.next
            index += 1
        print("Value not found")
    def sort(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp:
            curr = temp.next
```

```
            while curr:
                if temp.data > curr.data:
                    temp.data, curr.data = curr.data, temp.data

                curr = curr.next

            temp = temp.next

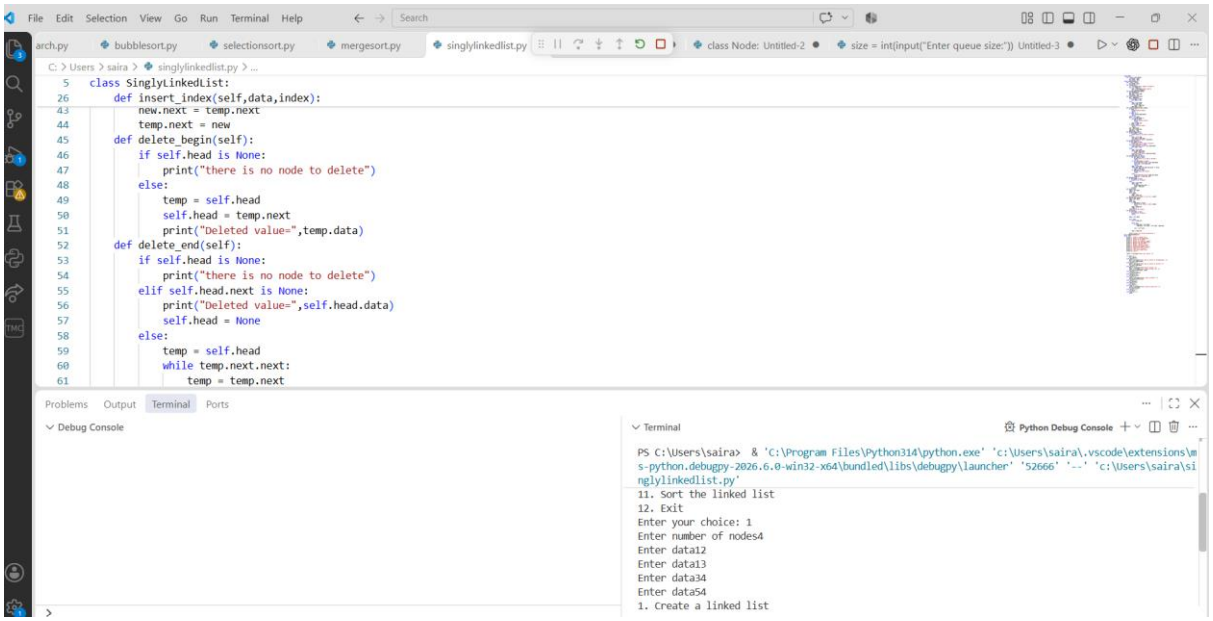
        print("Linked list sorted successfully.")
sll = SinglyLinkedList()
while True:
    print("1. Create a linked list")
    print("2. Insert at the beginning")
    print("3. Insert at the end")
    print("4. Insert at a specific index")
    print("5. Delete from the beginning")
    print("6. Delete from the end")
    print("7. Delete a specific value")
    print("8. Display the linked list")
    print("9. Count the number of nodes")
    print("10. Search for a value")
    print("11. Sort the linked list")
    print("12. Exit")

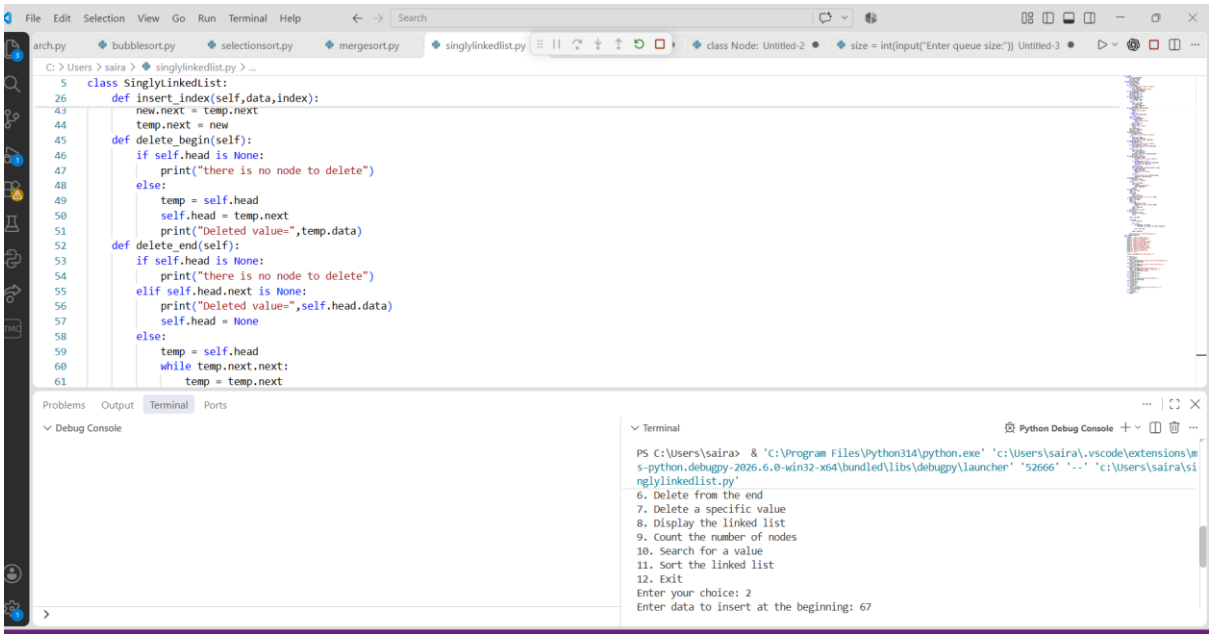
    choice = int(input("Enter your choice: "))
```

```
if choice == 1:
    sll.create()
elif choice == 2:
    data = int(input("Enter data to insert at the beginning: "))
    sll.insert_begin(data)
elif choice == 3:
    data = int(input("Enter data to insert at the end: "))
    sll.insert_end(data)
elif choice == 4:
    data = int(input("Enter data to insert: "))
    index = int(input("Enter index to insert at: "))
    sll.insert_index(data, index)
elif choice == 5:
    sll.delete_begin()
elif choice == 6:
    sll.delete_end()
elif choice == 7:
    value = int(input("Enter value to delete: "))
    sll.delete_value(value)
elif choice == 8:
    sll.display()
elif choice == 9:
    sll.count()
elif choice == 10:
    value = int(input("Enter value to search for: "))
    sll.search(value)
```

```
elif choice == 11:
    sll.sort()
elif choice == 12:
    break
```

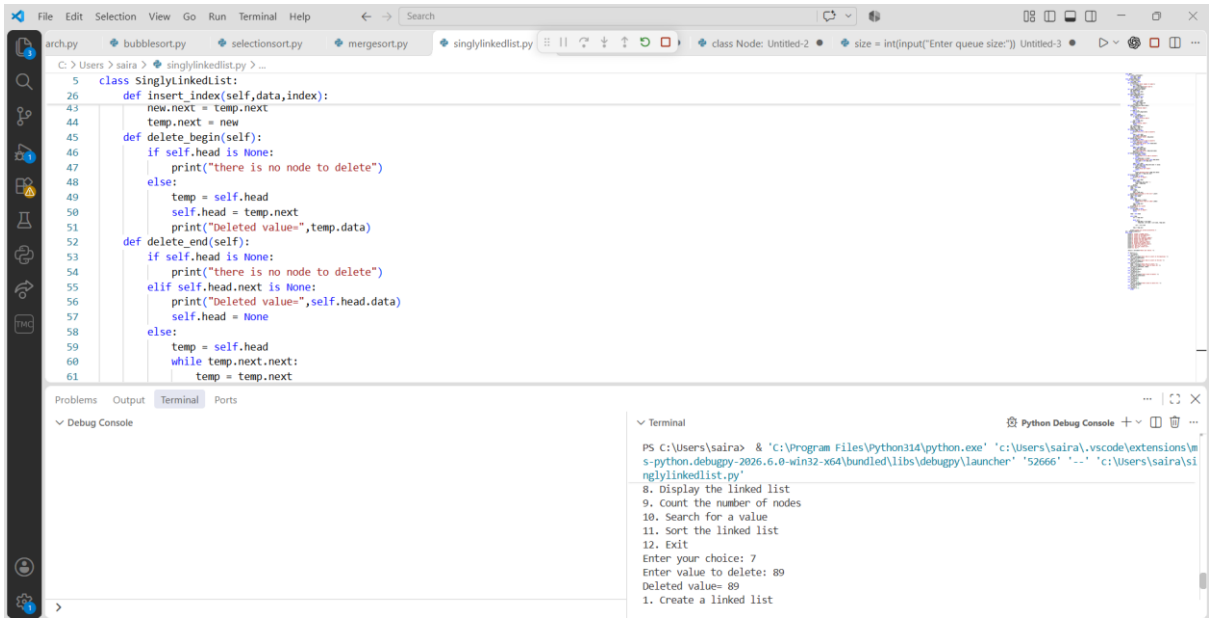
OUTPUT:





```
class SinglyLinkedList:
    def insert_index(self,data,index):
        new.next = temp.next
        temp.next = new
    def delete_begin(self):
        if self.head is None:
            print("there is no node to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted value=",temp.data)
    def delete_end(self):
        if self.head is None:
            print("there is no node to delete")
        elif self.head.next is None:
            print("Deleted value=",self.head.data)
            self.head = None
        else:
            temp = self.head
            while temp.next.next:
                temp = temp.next
```

```
PS C:\Users\saira> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52666' '--' 'c:\Users\saira\singlylinkedlist.py'
6. Delete from the end
7. Delete a specific value
8. Display the linked list
9. Count the number of nodes
10. Search for a value
11. Sort the linked list
12. Exit
Enter your choice: 2
Enter data to insert at the beginning: 67
```



```
class SinglyLinkedList:
    def insert_index(self,data,index):
        new.next = temp.next
        temp.next = new
    def delete_begin(self):
        if self.head is None:
            print("there is no node to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted value=",temp.data)
    def delete_end(self):
        if self.head is None:
            print("there is no node to delete")
        elif self.head.next is None:
            print("Deleted value=",self.head.data)
            self.head = None
        else:
            temp = self.head
            while temp.next.next:
                temp = temp.next
```

```
PS C:\Users\saira> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52666' '--' 'c:\Users\saira\singlylinkedlist.py'
8. Display the linked list
9. Count the number of nodes
10. Search for a value
11. Sort the linked list
12. Exit
Enter your choice: 7
Enter value to delete: 89
Deleted value= 89
1. Create a linked list
```

PROGRAM-6:

AIM: Write a python program to implement a Double Linked List with the below operations.

1) Create Linked List
2) Insert Beginning
3) Insert End
4) Insert at Index
5) Delete by Value
6) Delete First Node
7) Delete Last Node
8) Count number of nodes
9) Display or Traverse

Exit

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
```

```
def create(self):
    n = int(input("Enter number of nodes: "))
    for i in range(n):
        data = int(input("Enter data: "))
        self.insert_end(data)

def insert_beginning(self, data):
    new_node = Node(data)

    if self.head is not None:
        new_node.next = self.head
        self.head.prev = new_node

    self.head = new_node
    print("Node inserted at beginning")

def insert_end(self, data):

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    temp = self.head
    while temp.next:
```

```
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp
        print("Node inserted at end")

    def insert_at_index(self, data, index):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_beginning(data)
            return

        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid index")
                return
            temp = temp.next

        if temp is None:
            print("Invalid index")
            return
```

```
        new_node = Node(data)
        new_node.next = temp.next
        new_node.prev = temp

        if temp.next:
            temp.next.prev = new_node

        temp.next = new_node
        print("Node inserted at index", index)

    def delete_by_value(self, value):
        temp = self.head

        while temp and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not found")
            return

        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next

        if temp.next:
```

```
            temp.next.prev = temp.prev

        print("Node deleted")

    def delete_first(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

        if self.head:
            self.head.prev = None

        print("First node deleted")

    def delete_last(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        if temp.next is None:
            self.head = None
            print("Last node deleted")
```

```
            return

        while temp.next:
            temp = temp.next

        temp.prev.next = None
        print("Last node deleted")

    def count_nodes(self):
        count = 0
        temp = self.head

        while temp:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head
        print("Doubly Linked List:", end=" ")

        while temp:
```

```
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")

dll = DoublyLinkedList()

while True:
    print("\n--- Doubly Linked List Menu ---")
    print("1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display or Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
```

```
        data = int(input("Enter data: "))
        dll.insert_beginning(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        dll.insert_end(data)

    elif choice == 4:
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))
        dll.insert_at_index(data, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        dll.delete_by_value(value)

    elif choice == 6:
        dll.delete_first()

    elif choice == 7:
        dll.delete_last()

    elif choice == 8:
        dll.count_nodes()

    elif choice == 9:
```

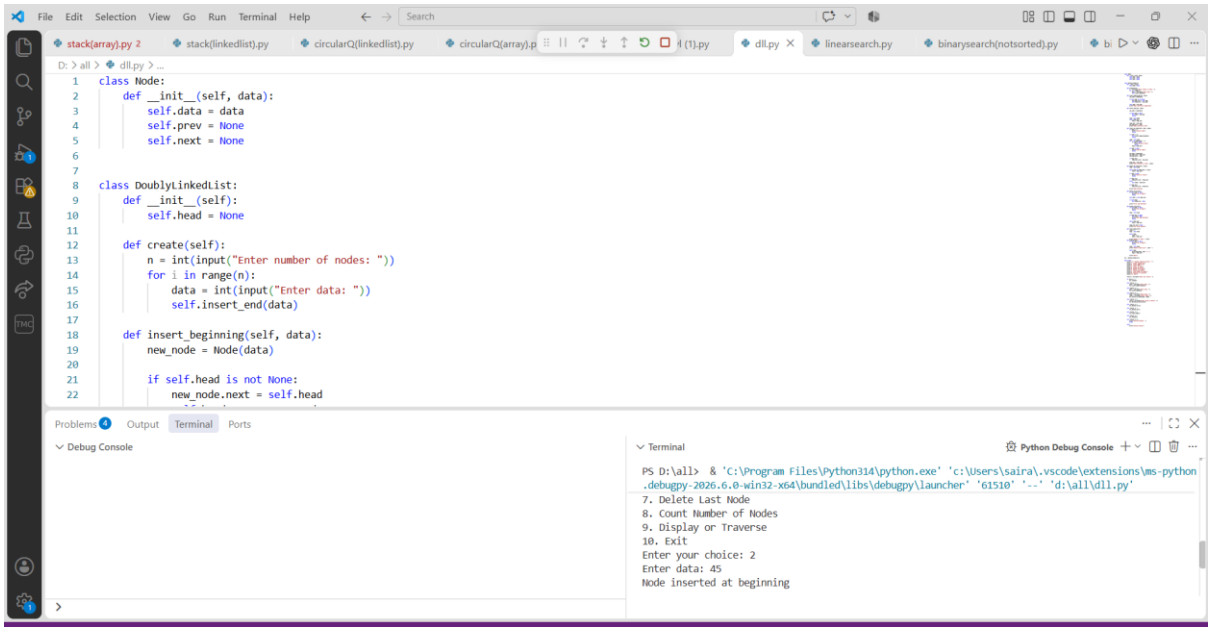
```
        dll.display()

    elif choice == 10:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:





```python
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

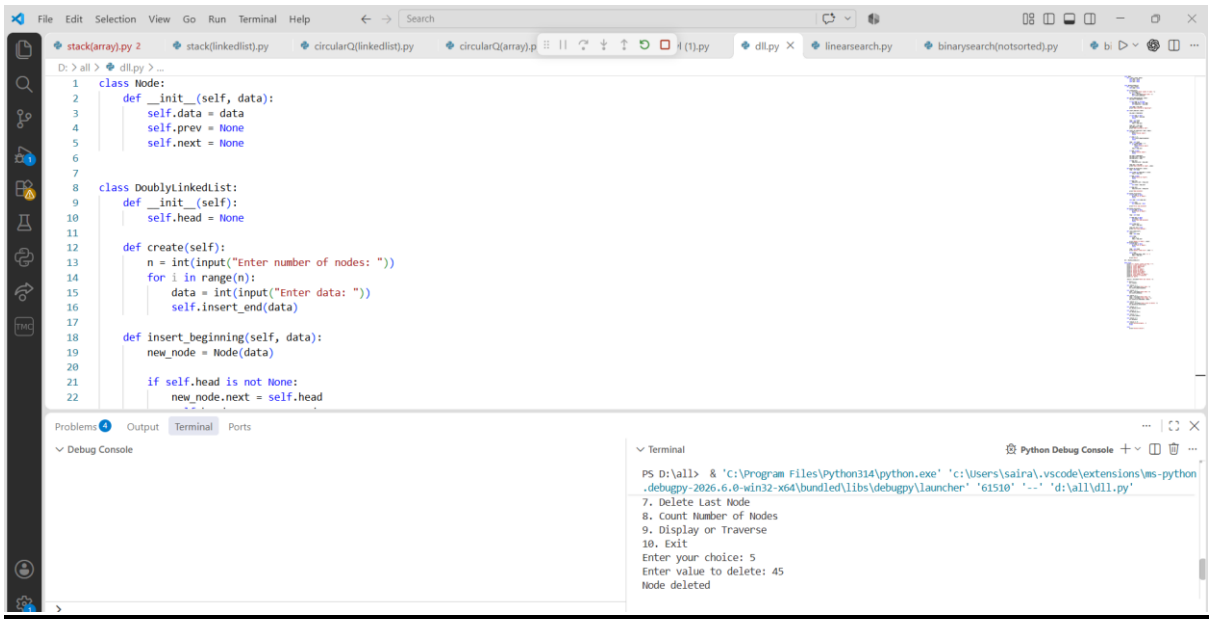

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter data: "))
            self.insert_end(data)

    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python
.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61510' '--' 'd:\all\dll.py'
7. Delete Last Node
8. Count Number of Nodes
9. Display or Traverse
10. Exit
Enter your choice: 2
Enter data: 45
Node inserted at beginning
```



```python
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

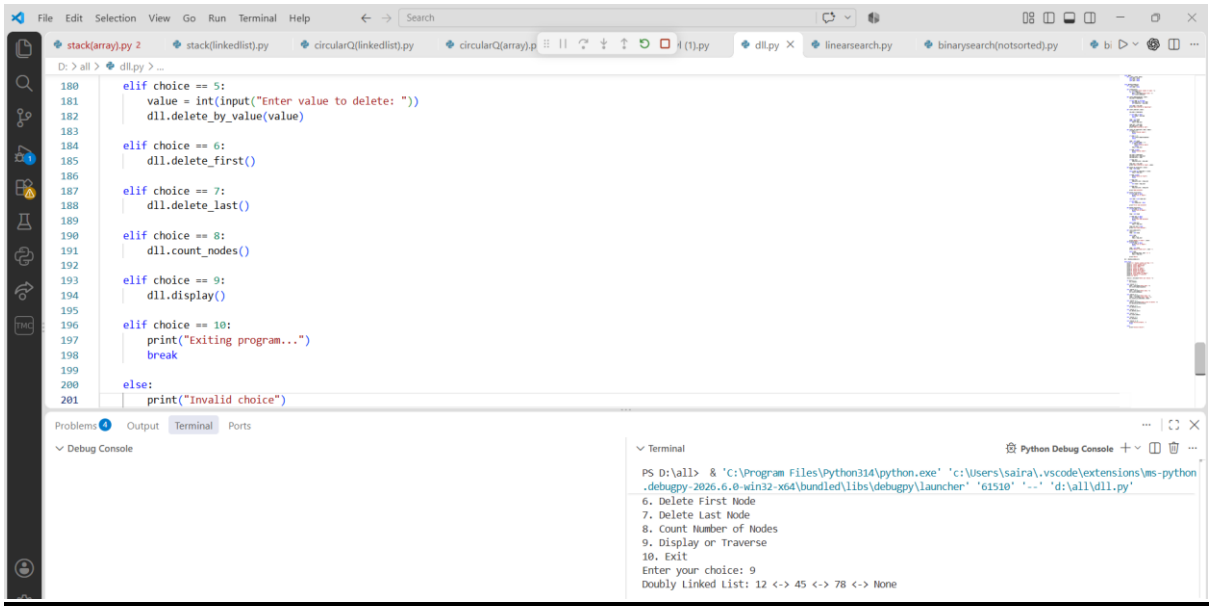

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input("Enter data: "))
            self.insert_end(data)

    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is not None:
            new_node.next = self.head
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python
.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61510' '--' 'd:\all\dll.py'
7. Delete Last Node
8. Count Number of Nodes
9. Display or Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 45
Node deleted
```


elif choice == 5:
value = int(input("Enter value to delete: "))
dll.delete_by_value(value)
elif choice == 6:
dll.delete_first()
elif choice == 7:
dll.delete_last()
elif choice == 8:
dll.count_nodes()
elif choice == 9:
dll.display()
elif choice == 10:
print("Exiting program...")
break
else:
print("Invalid choice")
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '61510' '--' 'd:\all\dll.py'
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display or Traverse
10. Exit
Enter your choice: 9
Doubly Linked List: 12 <-> 45 <-> 78 <-> None

PROGRAM-7(A):

AIM: Write a python program to implement stack operations such as push, pop, peek, display using an array.

CODE:

```
stack = []
size = int(input("Enter stack size: "))

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(stack) == size:
            print("Stack Overflow")
        else:
            item = int(input("Enter element: "))
            stack.append(item)
            print("Element pushed")

    elif choice == 2:
```

```
        if len(stack) == 0:
            print("Stack Underflow")
        else:
            print("Popped element:", stack.pop())

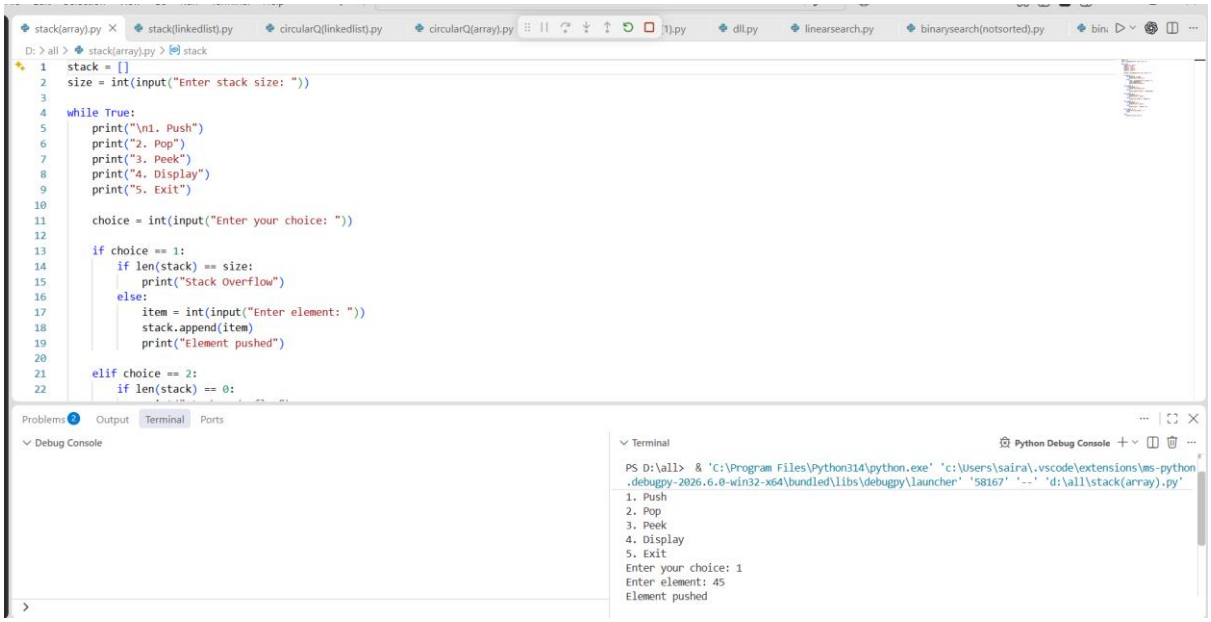
    elif choice == 3:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Top element:", stack[-1])

    elif choice == 4:
        if len(stack) == 0:
            print("Stack is empty")
        else:
            print("Stack:", stack[::-1])

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



```
stack = []
size = int(input("Enter stack size: "))

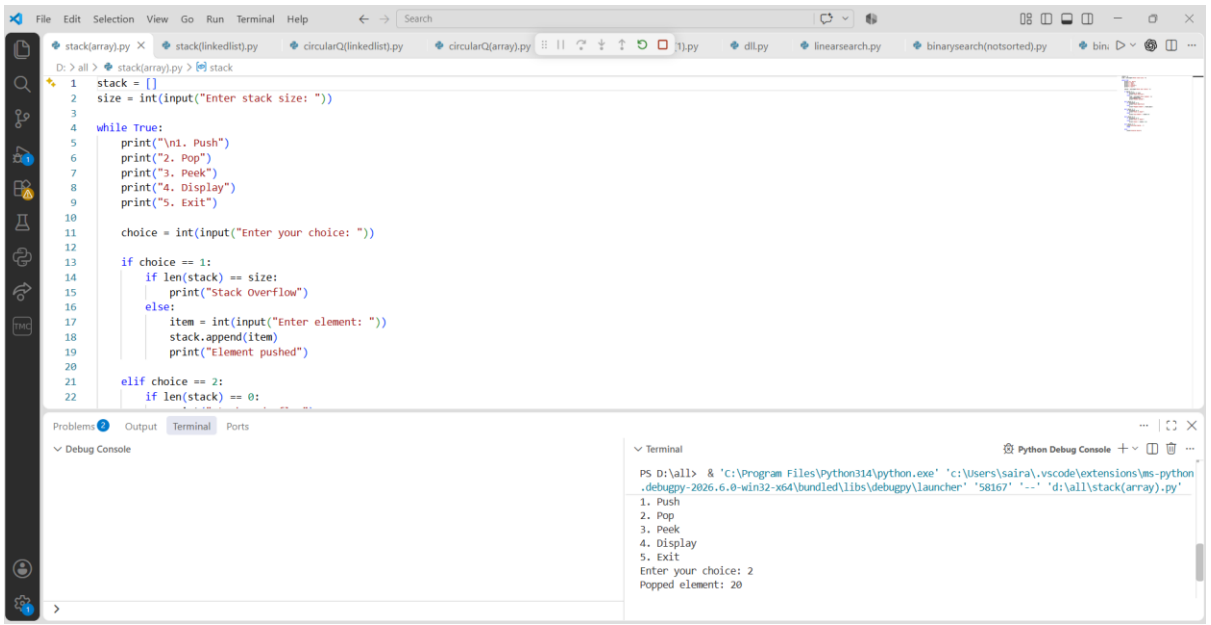
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(stack) == size:
            print("Stack Overflow")
        else:
            item = int(input("Enter element: "))
            stack.append(item)
            print("Element pushed")

    elif choice == 2:
        if len(stack) == 0:
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python
.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58167' '--' 'd:\all\stack(array).py'
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 45
Element pushed
```



```
stack = []
size = int(input("Enter stack size: "))

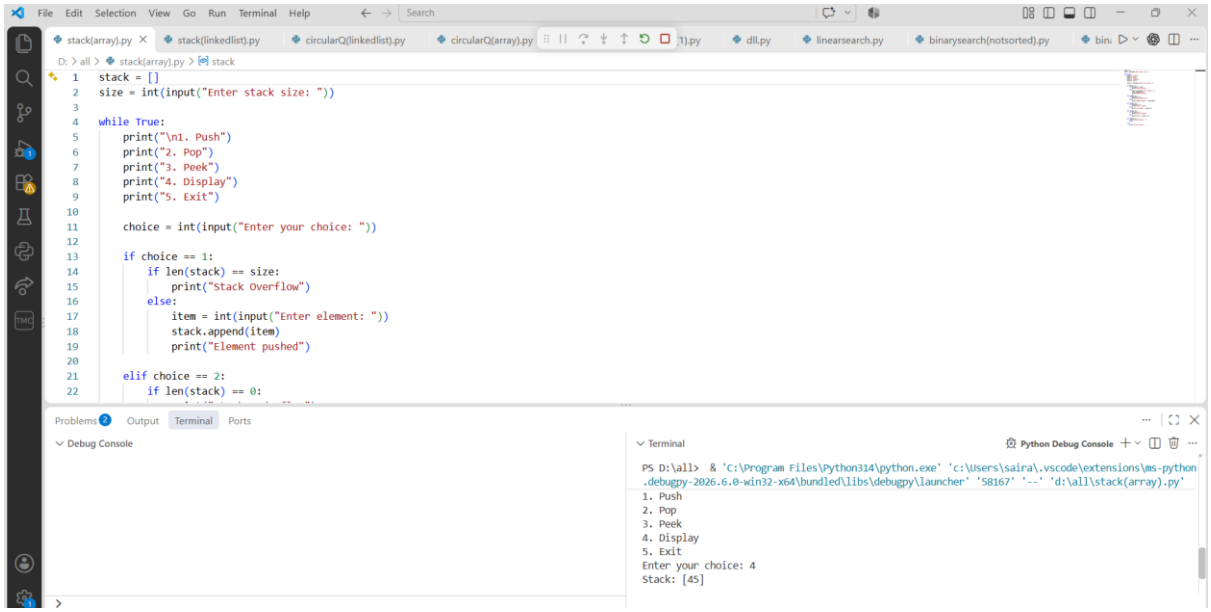
while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(stack) == size:
            print("Stack Overflow")
        else:
            item = int(input("Enter element: "))
            stack.append(item)
            print("Element pushed")

    elif choice == 2:
        if len(stack) == 0:
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python
.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58167' '--' 'd:\all\stack(array).py'
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 20
```


File Edit Selection View Go Run Terminal Help
stack(array).py
stack(linkedlist).py
circularQ(linkedlist).py
circularQ(array).py
dll.py
linearsearch.py
binarysearch(notsorted).py
stack = []
size = int(input("Enter stack size: "))
while True:
print("\n1. Push")
print("2. Pop")
print("3. Peek")
print("4. Display")
print("5. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
if len(stack) == size:
print("Stack Overflow")
else:
item = int(input("Enter element: "))
stack.append(item)
print("Element pushed")
elif choice == 2:
if len(stack) == 0:
Problems
Output
Terminal
Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58167' '--' 'd:\all\stack(array).py'
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack: [45]

PROGRAM-7(B):

AIM: Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None


    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print("Element pushed")


    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next
```

```
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack:", end=" ")
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()

s = Stack()

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
choice = int(input("Enter your choice: "))

if choice == 1:
    item = int(input("Enter element: "))
    s.push(item)

elif choice == 2:
    s.pop()

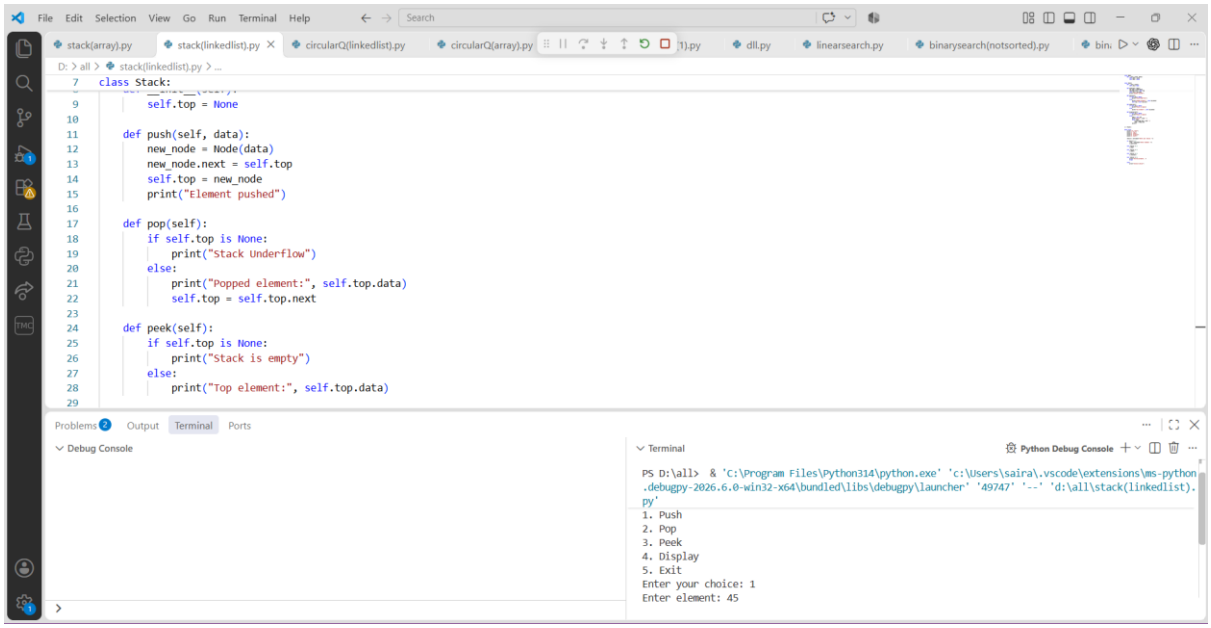
elif choice == 3:
    s.peek()

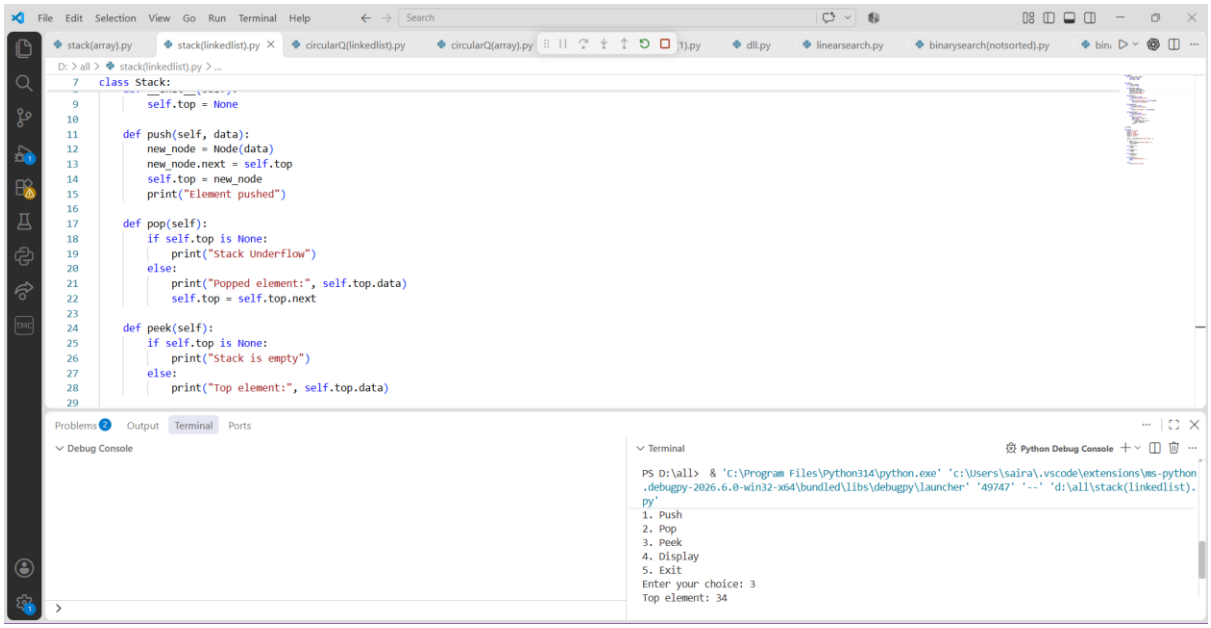
elif choice == 4:
    s.display()

elif choice == 5:
    print("Exiting program...")
    break

else:
    print("Invalid choice")
```

OUTPUT:


File Edit Selection View Go Run Terminal Help
Search
stack(array).py
stack(linkedlist).py
circularQ(linkedlist).py
circularQ(array).py
dll.py
linearsearch.py
binarysearch(notsorted).py
D: > all > stack(linkedlist).py > ...
class Stack:
self.top = None
def push(self, data):
new_node = Node(data)
new_node.next = self.top
self.top = new_node
print("Element pushed")
def pop(self):
if self.top is None:
print("Stack Underflow")
else:
print("Popped element:", self.top.data)
self.top = self.top.next
def peek(self):
if self.top is None:
print("Stack is empty")
else:
print("Top element:", self.top.data)
Problems 2
Output
Terminal
Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49747' '--' 'd:\all\stack(linkedlist).py'
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 45



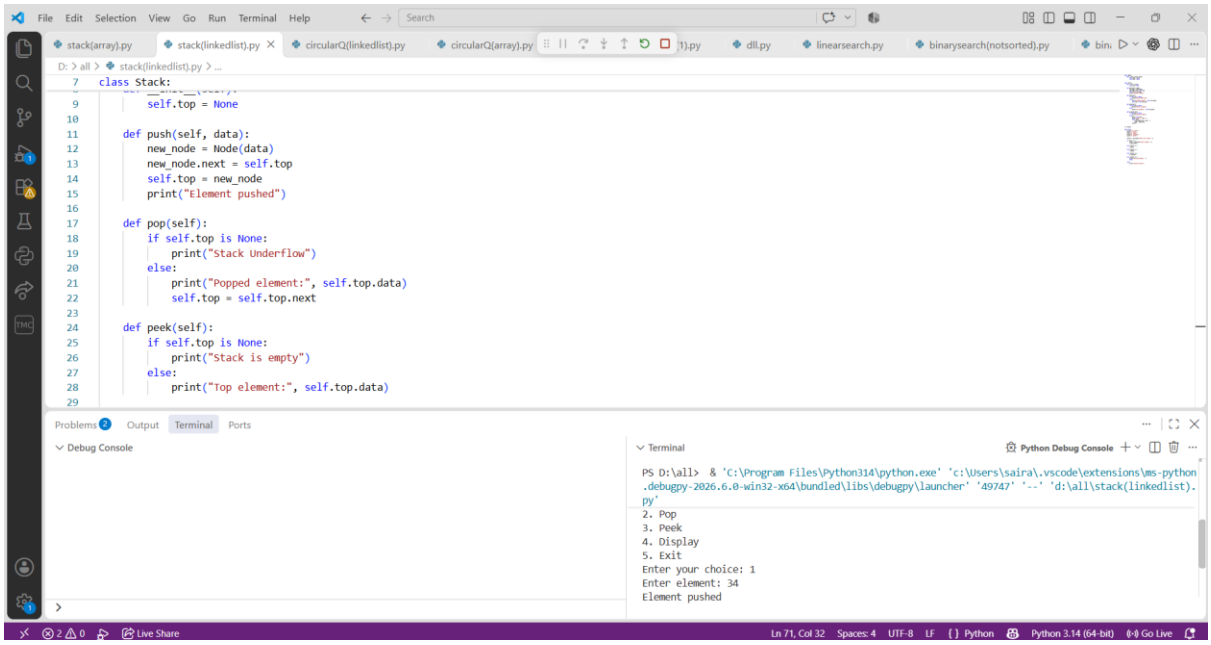
```
class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print("Element pushed")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49747' '--' 'd:\all\stack(linkedlist).py'
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 34
```



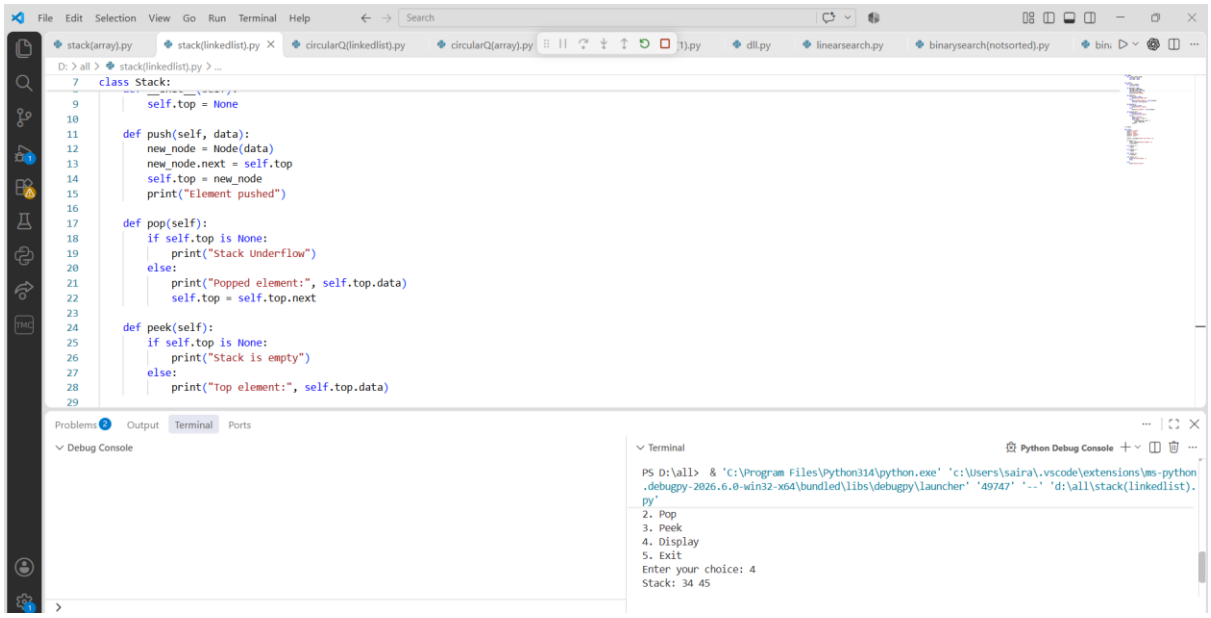
```
class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print("Element pushed")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            print("Popped element:", self.top.data)
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49747' '--' 'd:\all\stack(linkedlist).py'
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 34
Element pushed
```


File Edit Selection View Go Run Terminal Help
Search
stack(array).py
stack(linkedlist).py
circularQ(linkedlist).py
circularQ(array).py
dll.py
linearsearch.py
binarysearch(notsorted).py
D: > all > stack(linkedlist).py > ...
class Stack:
self.top = None
def push(self, data):
new_node = Node(data)
new_node.next = self.top
self.top = new_node
print("Element pushed")
def pop(self):
if self.top is None:
print("Stack Underflow")
else:
print("Popped element:", self.top.data)
self.top = self.top.next
def peek(self):
if self.top is None:
print("Stack is empty")
else:
print("Top element:", self.top.data)
Problems 2
Output
Terminal
Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49747' '--' 'd:\all\stack(linkedlist).py'
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack: 34 45

PROGRAM-8(A):

AIM: Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

Using Array

CODE:

```
queue = []
size = int(input("Enter queue size: "))


while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(queue) == size:
            print("Queue Overflow")
        else:
            item = int(input("Enter element: "))
            queue.append(item)
            print("Element inserted")
```

```
    elif choice == 2:
        if len(queue) == 0:
            print("Queue Underflow")
        else:
            print("Deleted element:", queue.pop(0))

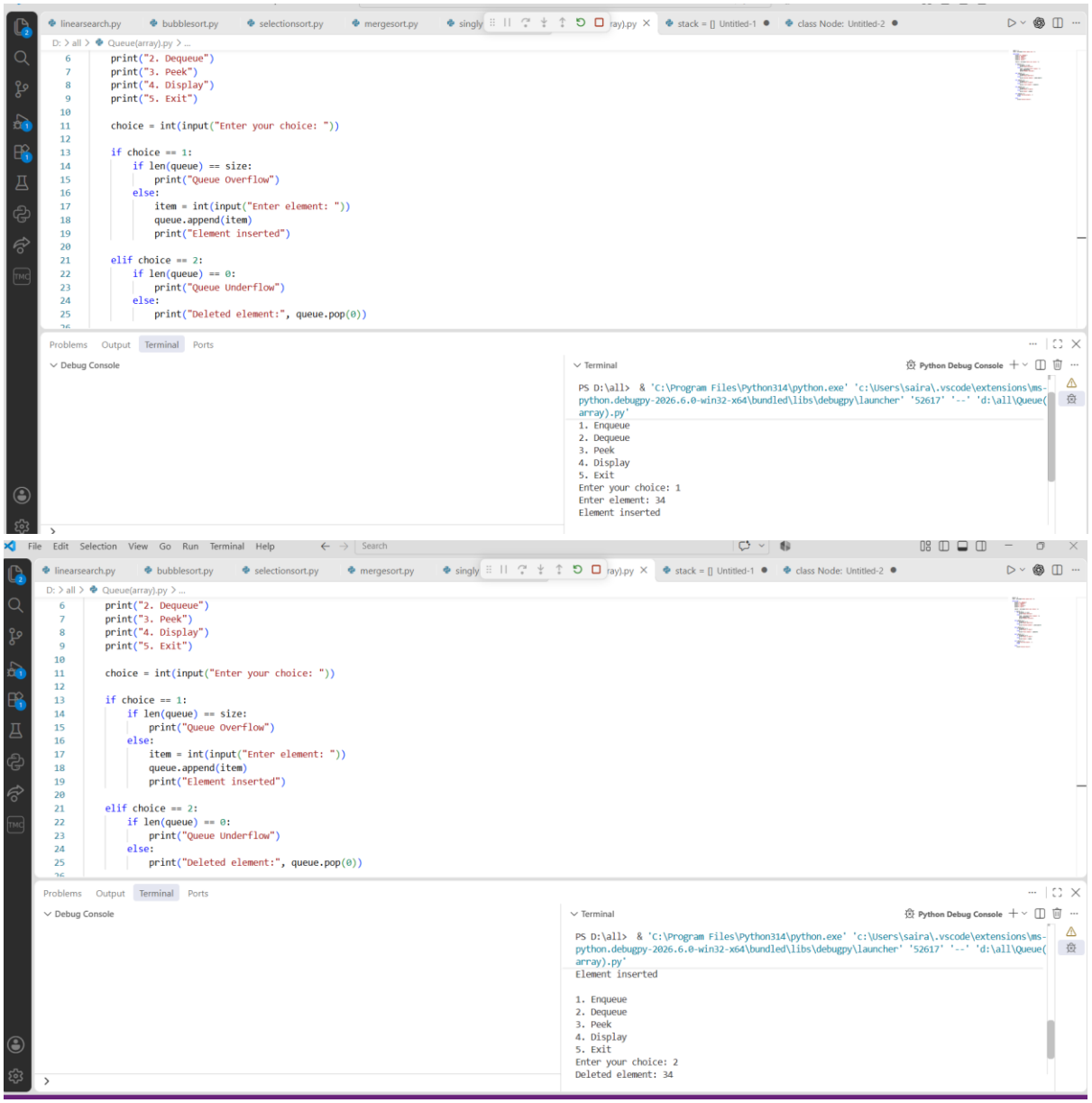
    elif choice == 3:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Front element:", queue[0])

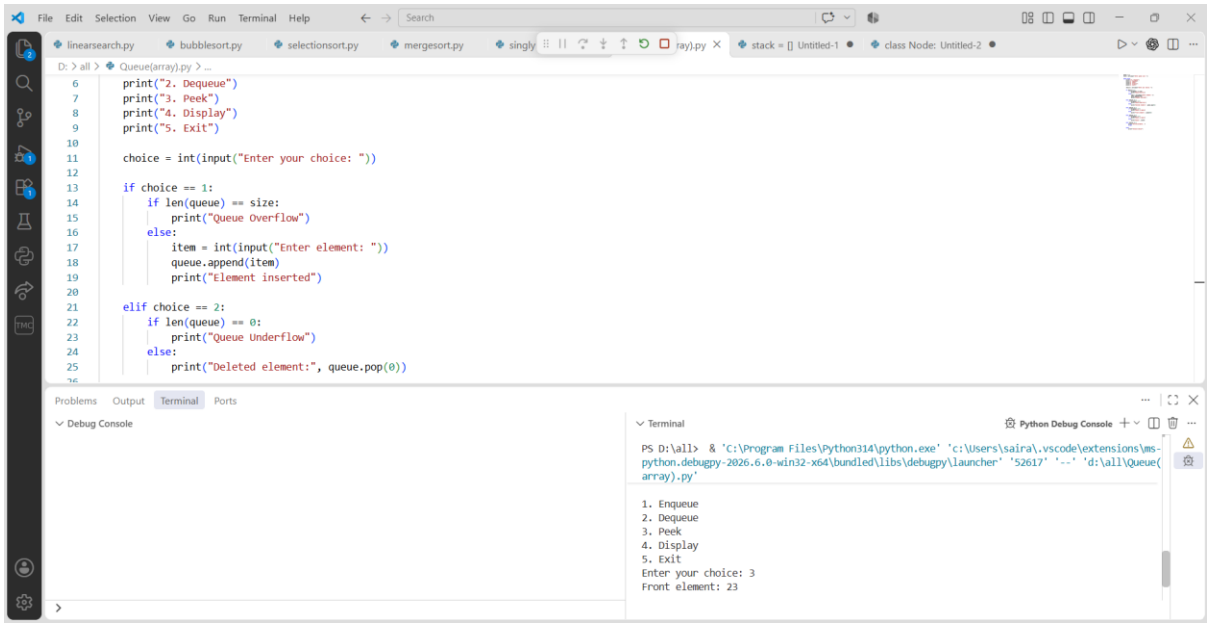
    elif choice == 4:
        if len(queue) == 0:
            print("Queue is empty")
        else:
            print("Queue:", queue)

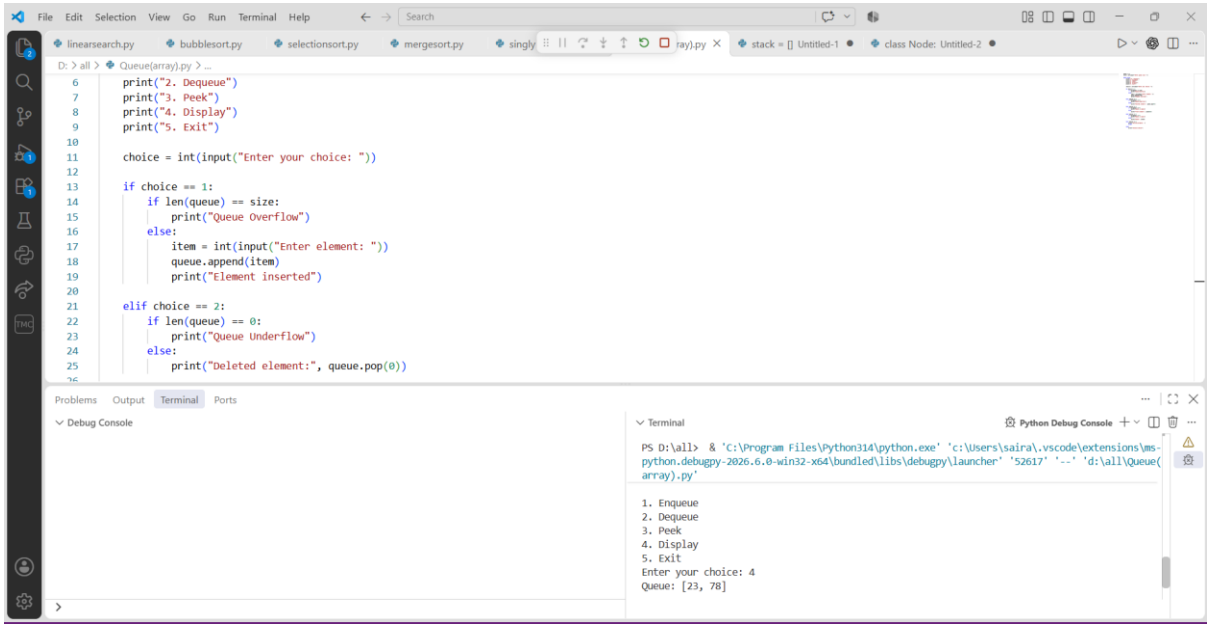
    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:


linearsearch.py
bubblesort.py
selectionsort.py
mergesort.py
stack = [] Untitled-1
class Node: Untitled-2
D: > all > Queue(array).py > ...
print("2. Dequeue")
print("3. Peek")
print("4. Display")
print("5. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
if len(queue) == size:
print("Queue Overflow")
else:
item = int(input("Enter element: "))
queue.append(item)
print("Element inserted")
elif choice == 2:
if len(queue) == 0:
print("Queue Underflow")
else:
print("Deleted element:", queue.pop(0))
Problems
Output
Terminal
Ports
Debug Console
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52617' '--' 'd:\all\Queue(array).py'
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 34
Element inserted
File
Edit
Selection
View
Go
Run
Terminal
Help
Search
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52617' '--' 'd:\all\Queue(array).py'
Element inserted
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 34


File Edit Selection View Go Run Terminal Help
linearsearch.py
bubblesort.py
selectionsort.py
mergesort.py
stack = [] Untitled-1
class Node: Untitled-2
print("2. Dequeue")
print("3. Peek")
print("4. Display")
print("5. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
if len(queue) == size:
print("Queue Overflow")
else:
item = int(input("Enter element: "))
queue.append(item)
print("Element inserted")
elif choice == 2:
if len(queue) == 0:
print("Queue Underflow")
else:
print("Deleted element:", queue.pop(0))
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52617' '--' 'd:\all\Queue(array).py'
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 23


File Edit Selection View Go Run Terminal Help
linearsearch.py
bubblesort.py
selectionsort.py
mergesort.py
stack = [] Untitled-1
class Node: Untitled-2
print("2. Dequeue")
print("3. Peek")
print("4. Display")
print("5. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
if len(queue) == size:
print("Queue Overflow")
else:
item = int(input("Enter element: "))
queue.append(item)
print("Element inserted")
elif choice == 2:
if len(queue) == 0:
print("Queue Underflow")
else:
print("Deleted element:", queue.pop(0))
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52617' '--' 'd:\all\Queue(array).py'
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue: [23, 78]

Using Linked List:

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print("Element inserted")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
```

```
    else:
        print("Deleted element:", self.front.data)
        self.front = self.front.next

        if self.front is None:
            self.rear = None

def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)

def display(self):
    if self.front is None:
        print("Queue is empty")
    else:
        temp = self.front
        print("Queue:", end=" ")

        while temp:
            print(temp.data, end=" ")
            temp = temp.next

        print()
```

```
q = Queue()

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

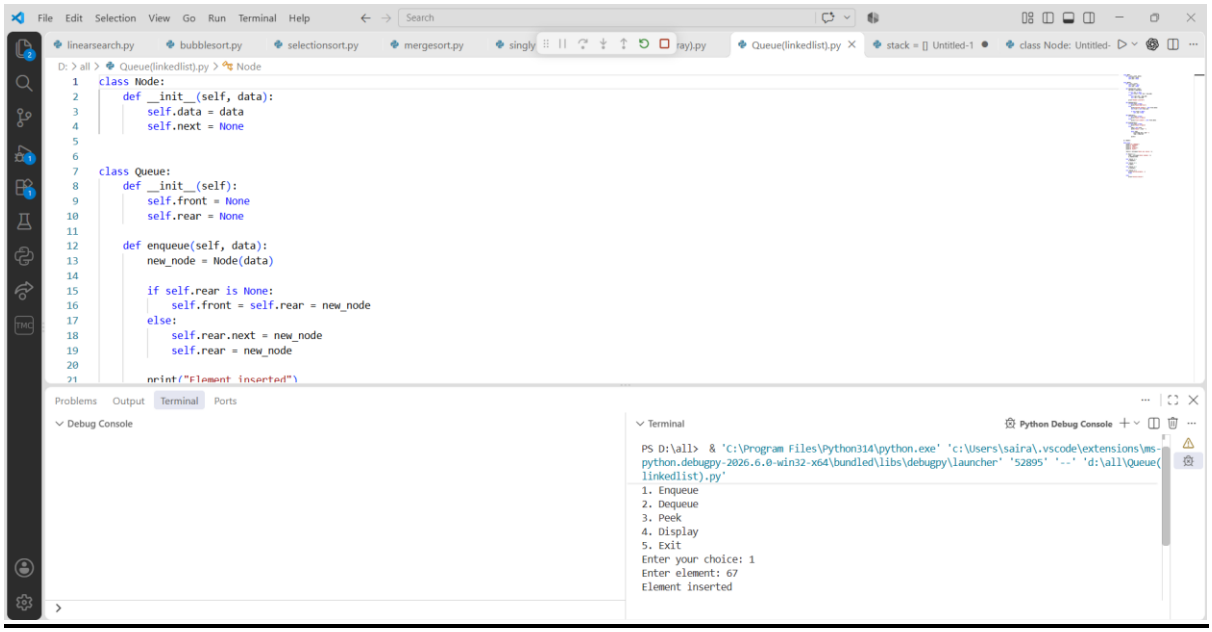
    elif choice == 4:
        q.display()

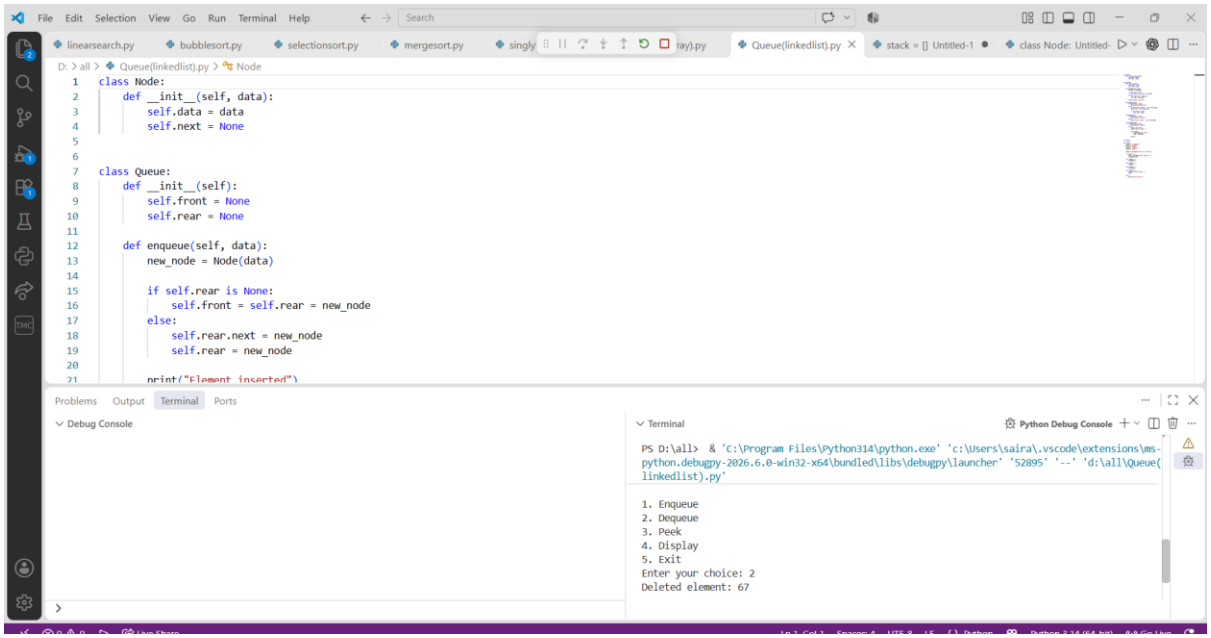
    elif choice == 5:
        print("Exiting program...")
```

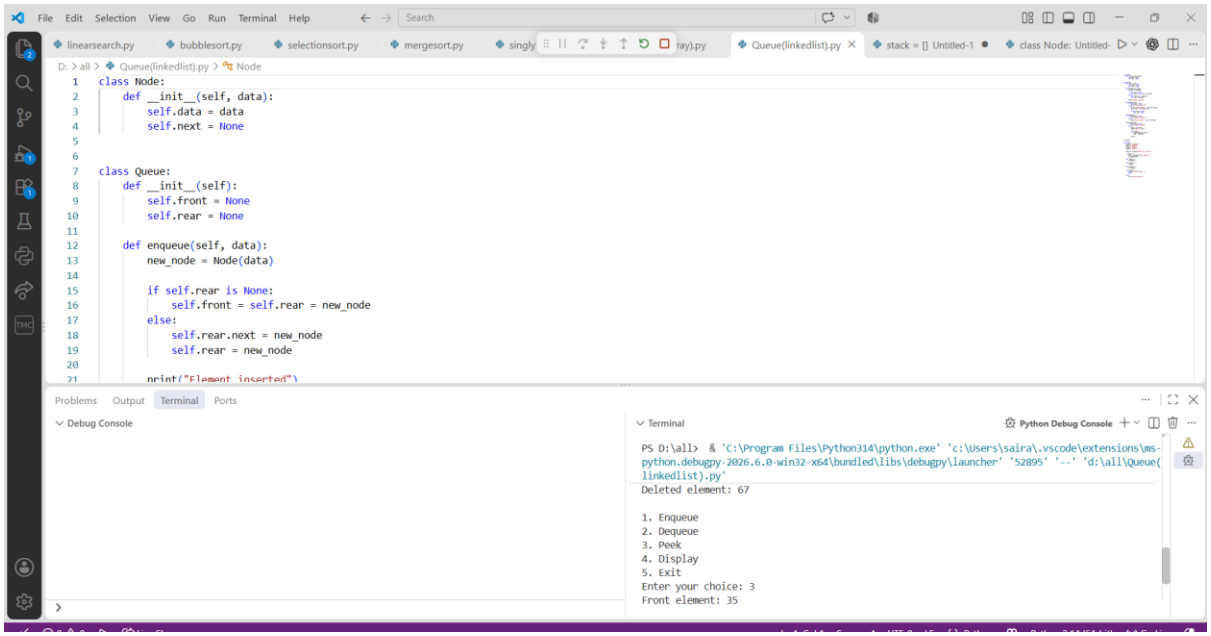
```
        break

    else:
        print("Invalid choice")
```

OUTPUT:







```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

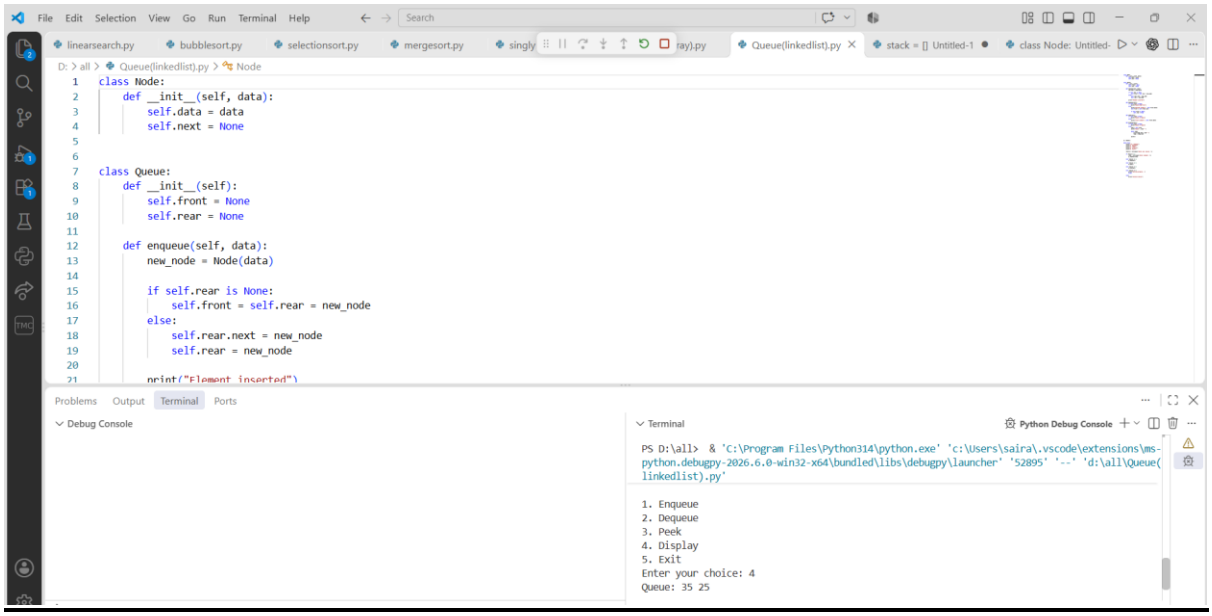
    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print("Element inserted")
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52895' '--' 'd:\all\Queue(linkedlist).py'
Deleted element: 67

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 35
```



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print("Element inserted")
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52895' '--' 'd:\all\Queue(linkedlist).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue: 35 25
```

PROGRAM-8(B):

AIM: Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

Using Array

CODE:

```
size = int(input("Enter queue size: "))
queue = [None] * size
front = -1
rear = -1

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if (rear + 1) % size == front:
            print("Queue Overflow")
        else:
            item = int(input("Enter element: "))
```

```
        if front == -1:
            front = 0

        rear = (rear + 1) % size
        queue[rear] = item
        print("Element inserted")

    elif choice == 2:
        if front == -1:
            print("Queue Underflow")
        else:
            print("Deleted element:", queue[front])

            if front == rear:
                front = rear = -1
            else:
                front = (front + 1) % size

    elif choice == 3:
        if front == -1:
            print("Queue is empty")
        else:
            print("Front element:", queue[front])

    elif choice == 4:
        if front == -1:
```

```
            print("Queue is empty")
        else:
            print("Circular Queue:", end=" ")
            i = front

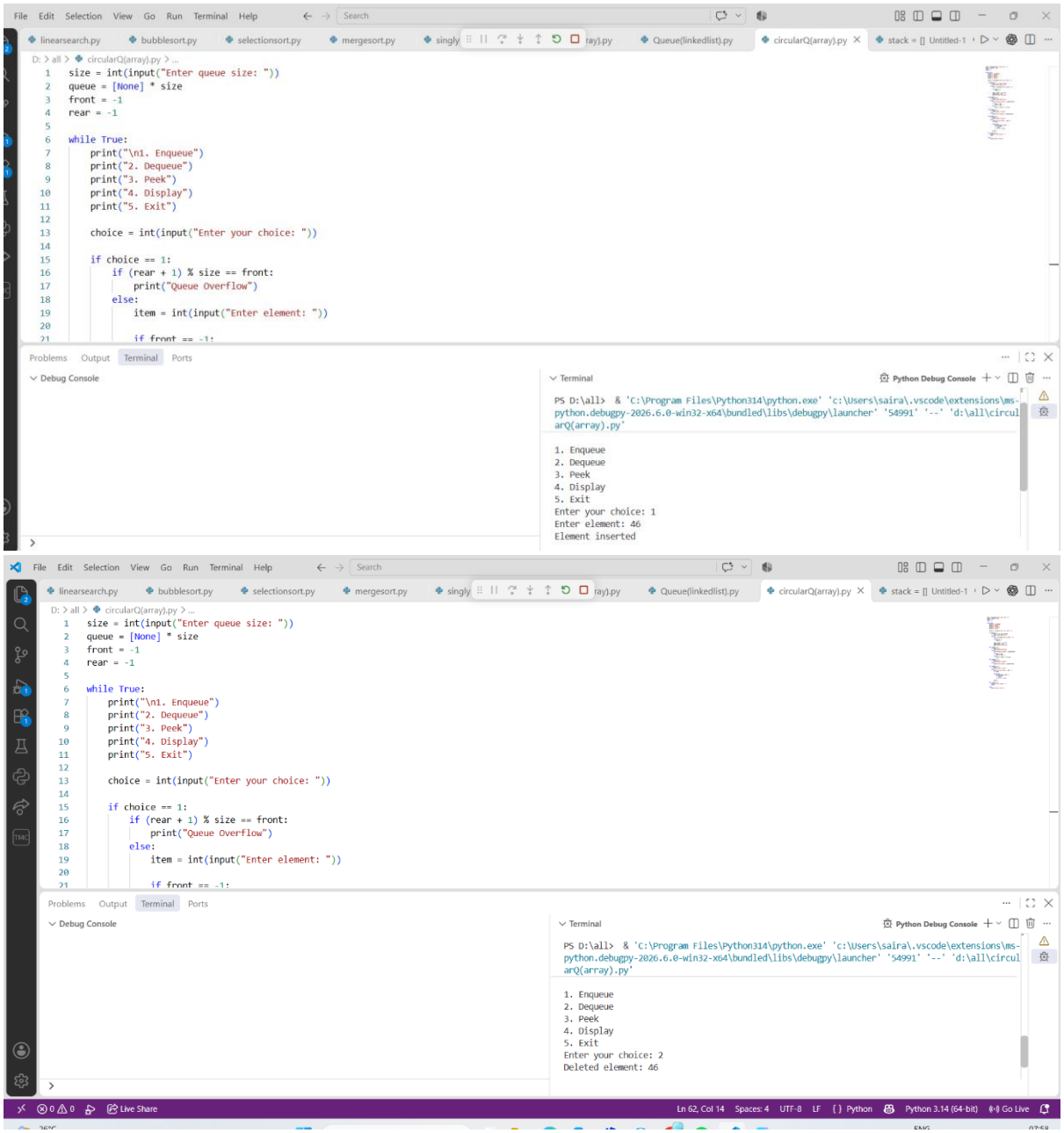
            while True:
                print(queue[i], end=" ")
                if i == rear:
                    break
                i = (i + 1) % size

            print()

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



```
size = int(input("Enter queue size: "))
queue = [None] * size
front = -1
rear = -1

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if (rear + 1) % size == front:
            print("Queue Overflow")
        else:
            item = int(input("Enter element: "))

            if front == -1:
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54991' '--' 'd:\all\circularQ(array).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 46
Element inserted
```



```
size = int(input("Enter queue size: "))
queue = [None] * size
front = -1
rear = -1

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

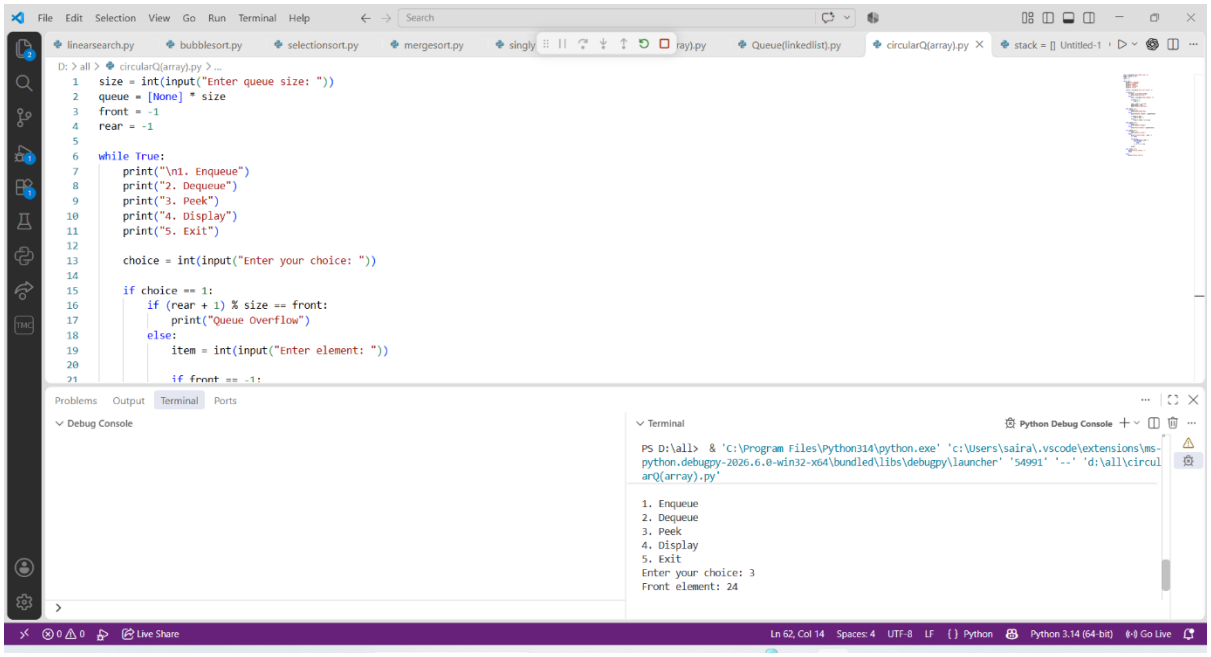
    choice = int(input("Enter your choice: "))

    if choice == 1:
        if (rear + 1) % size == front:
            print("Queue Overflow")
        else:
            item = int(input("Enter element: "))

            if front == -1:
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54991' '--' 'd:\all\circularQ(array).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 46
```



```
size = int(input("Enter queue size: "))
queue = [None] * size
front = -1
rear = -1

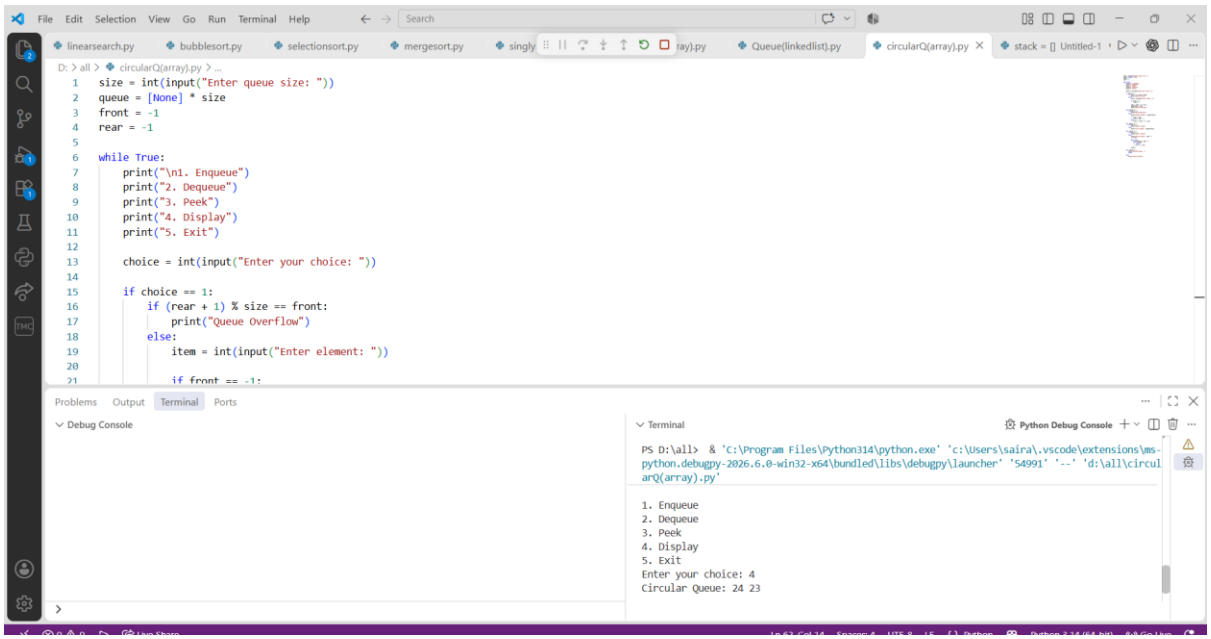
while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if (rear + 1) % size == front:
            print("Queue Overflow")
        else:
            item = int(input("Enter element: "))
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54991' '--' 'd:\all\circularQ(array).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 24
```



```
size = int(input("Enter queue size: "))
queue = [None] * size
front = -1
rear = -1

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if (rear + 1) % size == front:
            print("Queue Overflow")
        else:
            item = int(input("Enter element: "))
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54991' '--' 'd:\all\circularQ(array).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue: 24 23
```

Using Linked List

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.front is None:
            self.front = self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node

        print("Element inserted")

    def dequeue(self):
```

```
    if self.front is None:
        print("Queue Underflow")
    elif self.front == self.rear:
        print("Deleted element:", self.front.data)
        self.front = self.rear = None
    else:
        print("Deleted element:", self.front.data)
        self.front = self.front.next
        self.rear.next = self.front

def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)

def display(self):
    if self.front is None:
        print("Queue is empty")
    else:
        temp = self.front
        print("Circular Queue:", end=" ")

        while True:
            print(temp.data, end=" ")
            temp = temp.next
```

```
            if temp == self.front:
                break

        print()

q = CircularQueue()

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

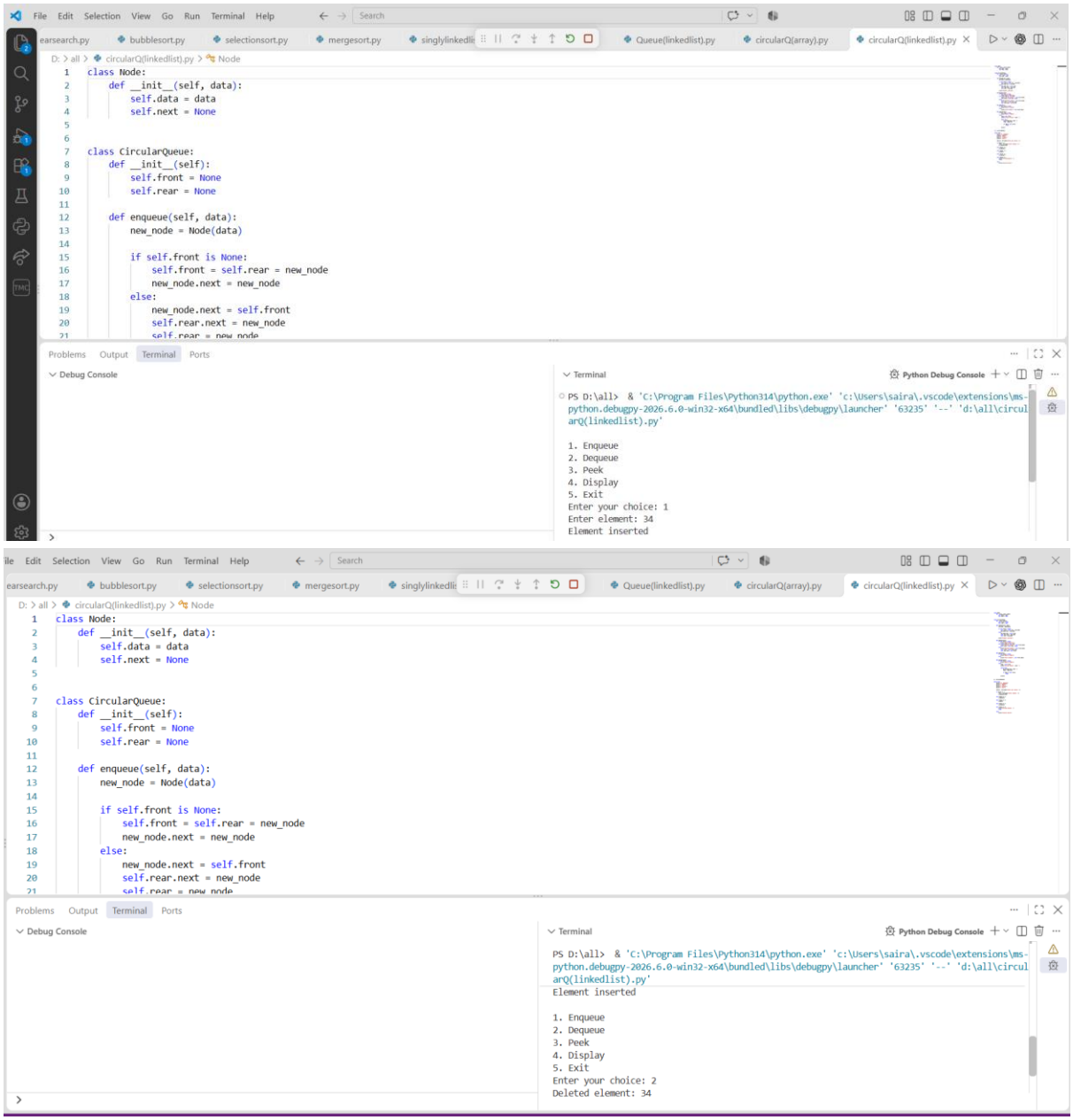
    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting program...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.front is None:
            self.front = self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63235' '--' 'd:\all\circularQ(linkedlist).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 34
Element inserted
```



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

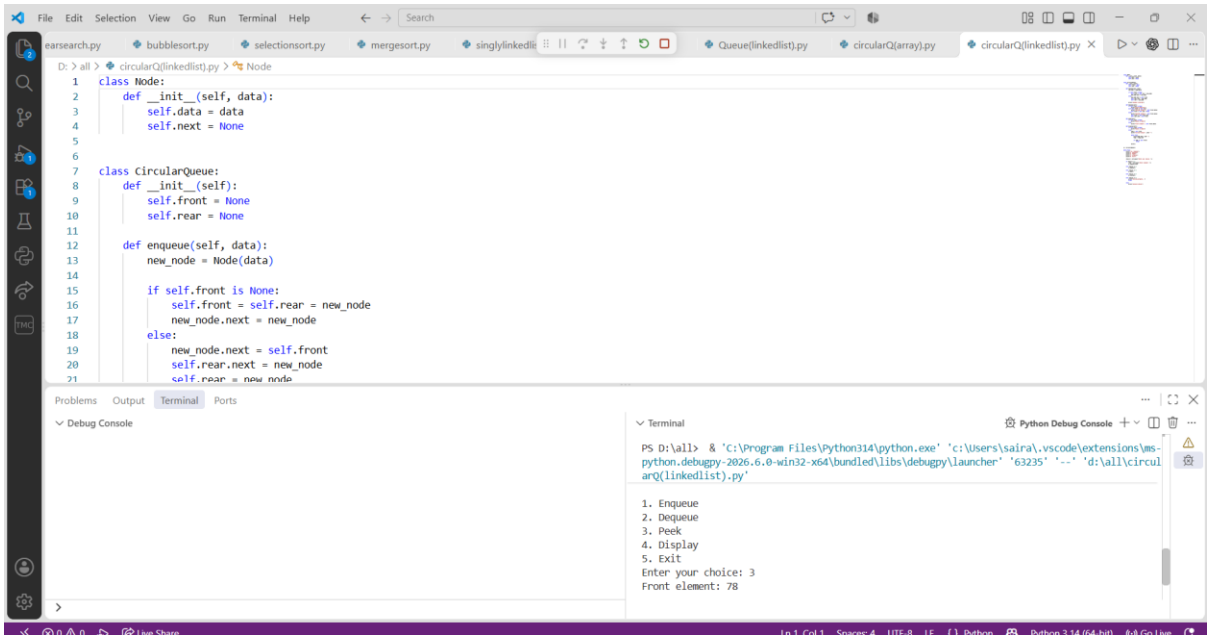

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.front is None:
            self.front = self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63235' '--' 'd:\all\circularQ(linkedlist).py'
Element inserted

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
Deleted element: 34
```



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

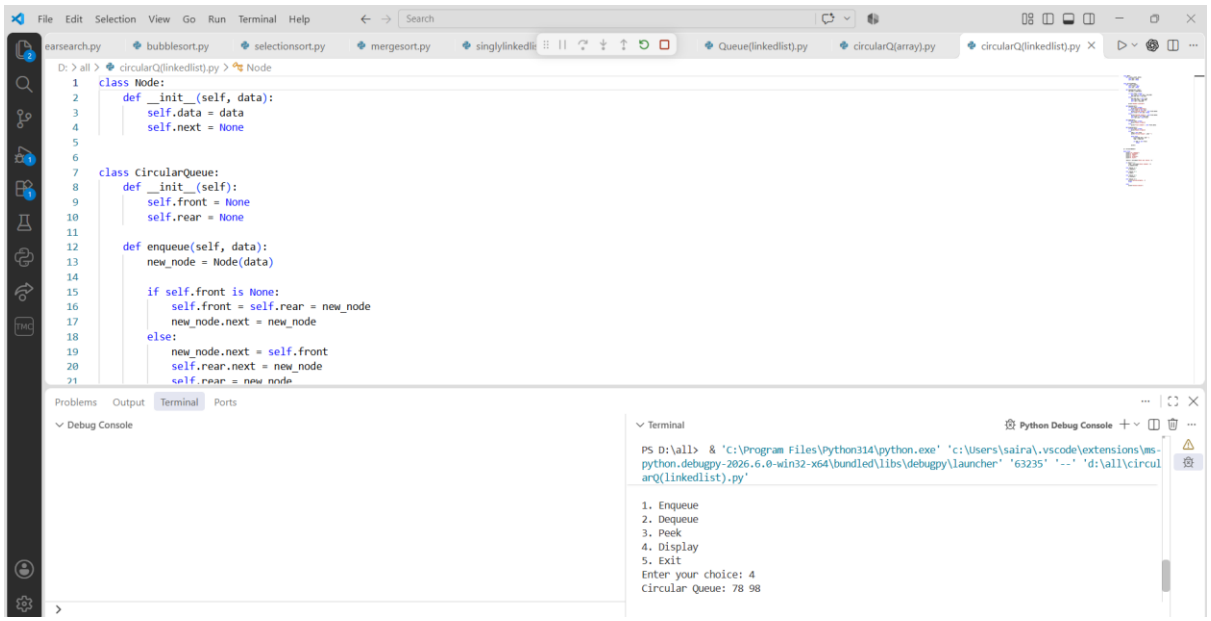

class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.front is None:
            self.front = self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63235' '--' 'd:\all\circularQ(linkedlist).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 78
```



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.front is None:
            self.front = self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63235' '--' 'd:\all\circularQ(linkedlist).py'

1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue: 78 98
```

PROGRAM-9:

AIM: Write a python program to implement a Binary Search Tree (BST) and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals – in-order, pre-order and post-order
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
```

```
    else:
        print("Duplicate value is not allowed.")
    return root

def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
```

```
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
```

```
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
    return max(left_height, right_height) + 1

def delete(root, x):
    if root is None:
        print("Element not found.")
```

```
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
```

```
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)

def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)

# ======================================================
# MAIN PROGRAM
# ======================================================

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
```

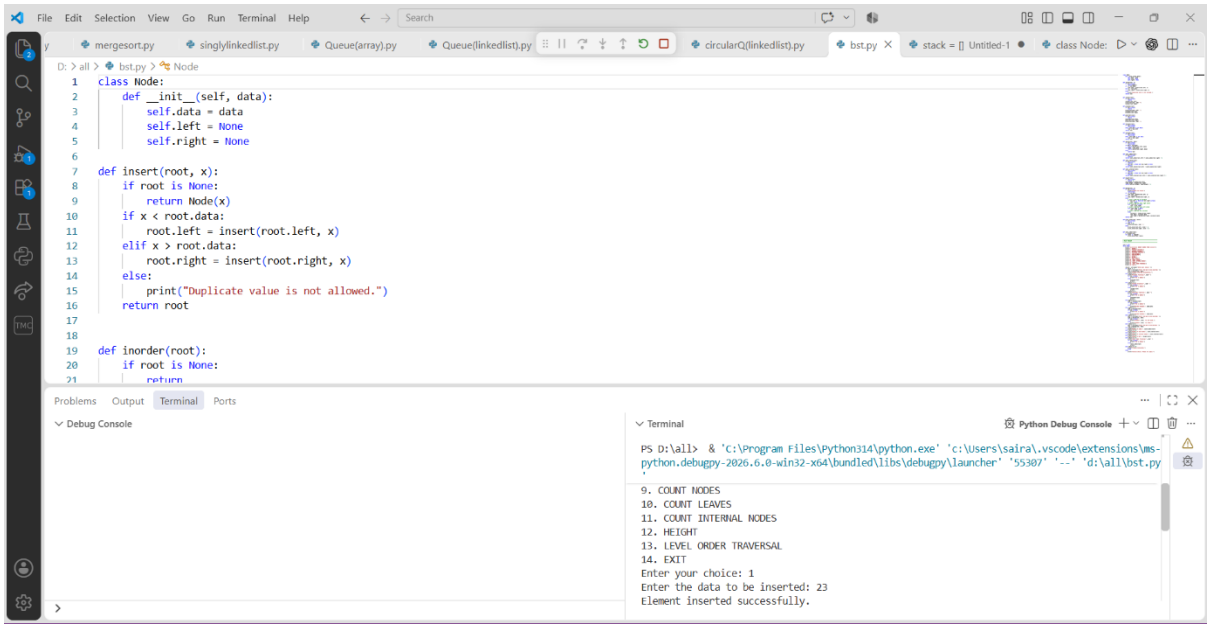
```
print("10. COUNT LEAVES")
print("11. COUNT INTERNAL NODES")
print("12. HEIGHT")
print("13. LEVEL ORDER TRAVERSAL")
print("14. EXIT")

choice = int(input("Enter your choice: "))
if choice == 1:
    data = int(input("Enter the data to be inserted: "))
    root = insert(root, data)
    print("Element inserted successfully.")
elif choice == 2:
    print("Preorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        preorder(root)
        print()
elif choice == 3:
    print("Inorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        inorder(root)
        print()
elif choice == 4:
```

```
        print("Postorder Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            postorder(root)
            print()
    elif choice == 5:
        node = find_min(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Minimum element:", node.data)
    elif choice == 6:
        node = find_max(root)
        if node is None:
            print("Tree is empty.")
        else:
            print("Maximum element:", node.data)
    elif choice == 7:
        data = int(input("Enter the data to be searched: "))
        node = search(root, data)
        if node is None:
            print("Element", data, "is not found.")
        else:
            print("Element", data, "is found.")
    elif choice == 8:
```

```
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 9:
        print("Number of nodes:", count_nodes(root))
    elif choice == 10:
        print("Number of leaf nodes:", count_leaves(root))
    elif choice == 11:
        print("Number of internal nodes:", count_internal(root))
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:

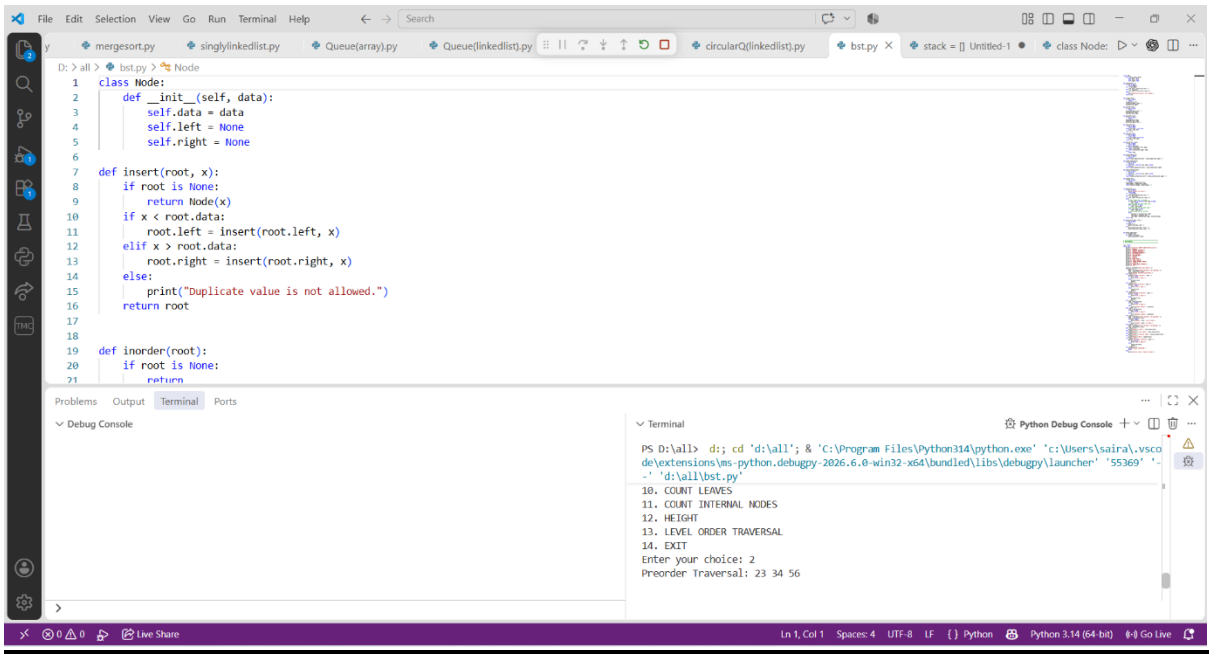


```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
```

```
PS D:\all>  & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55307' '--' 'd:\all\bst.py
'
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 1
Enter the data to be inserted: 23
Element inserted successfully.
```

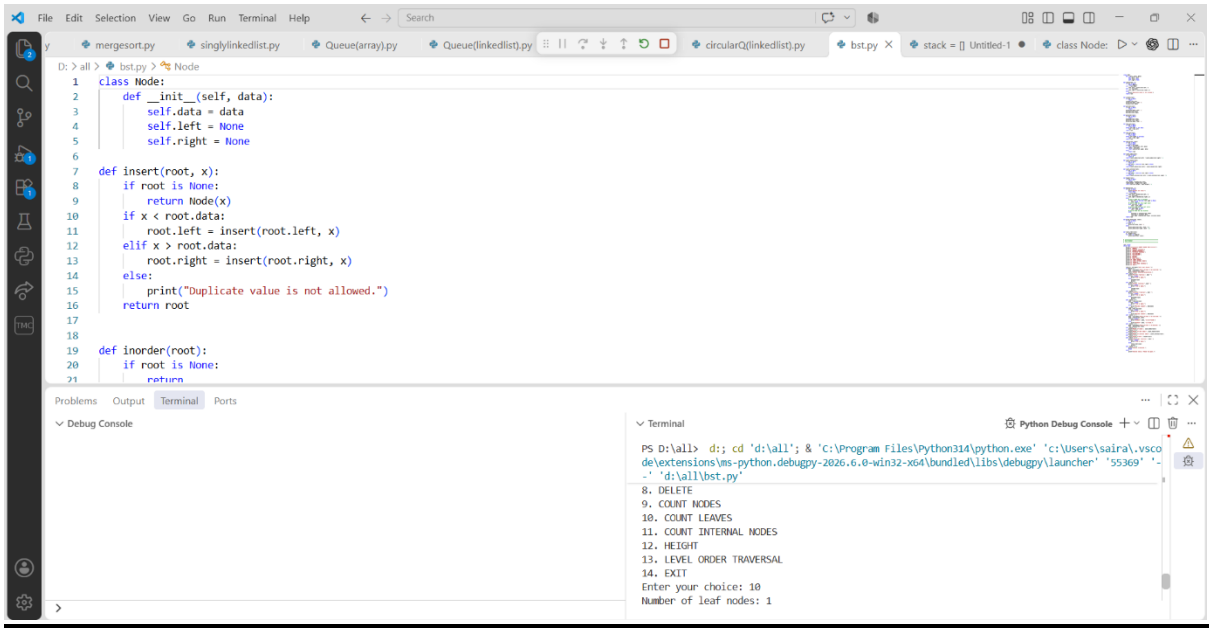


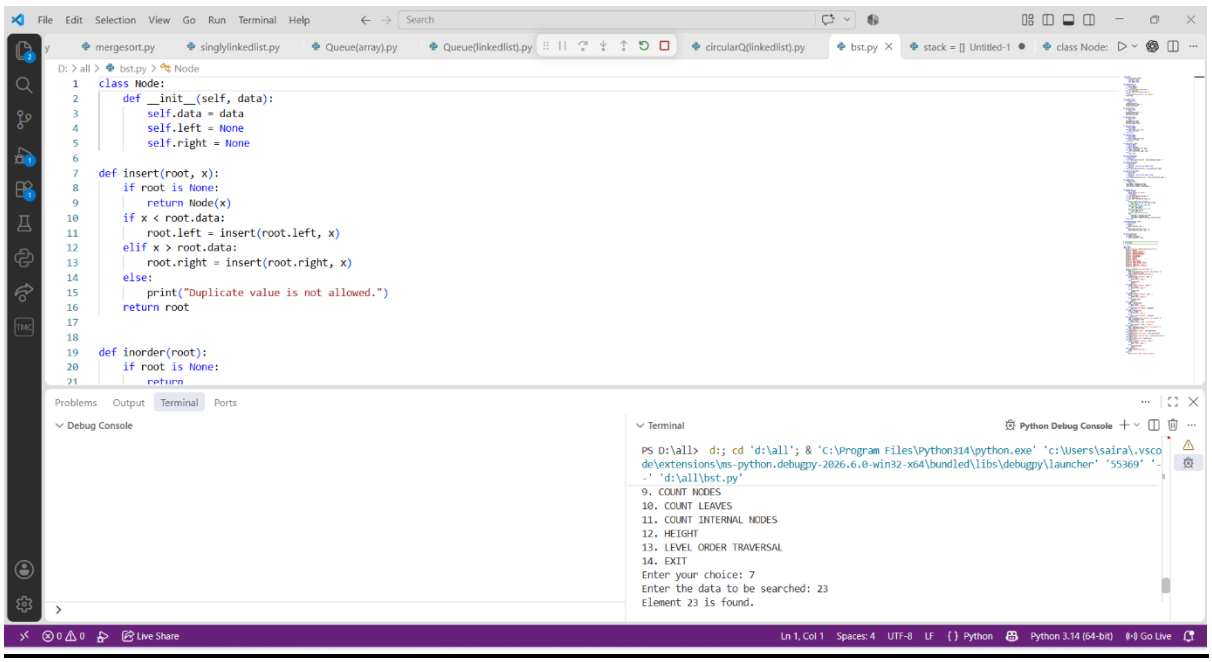
```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

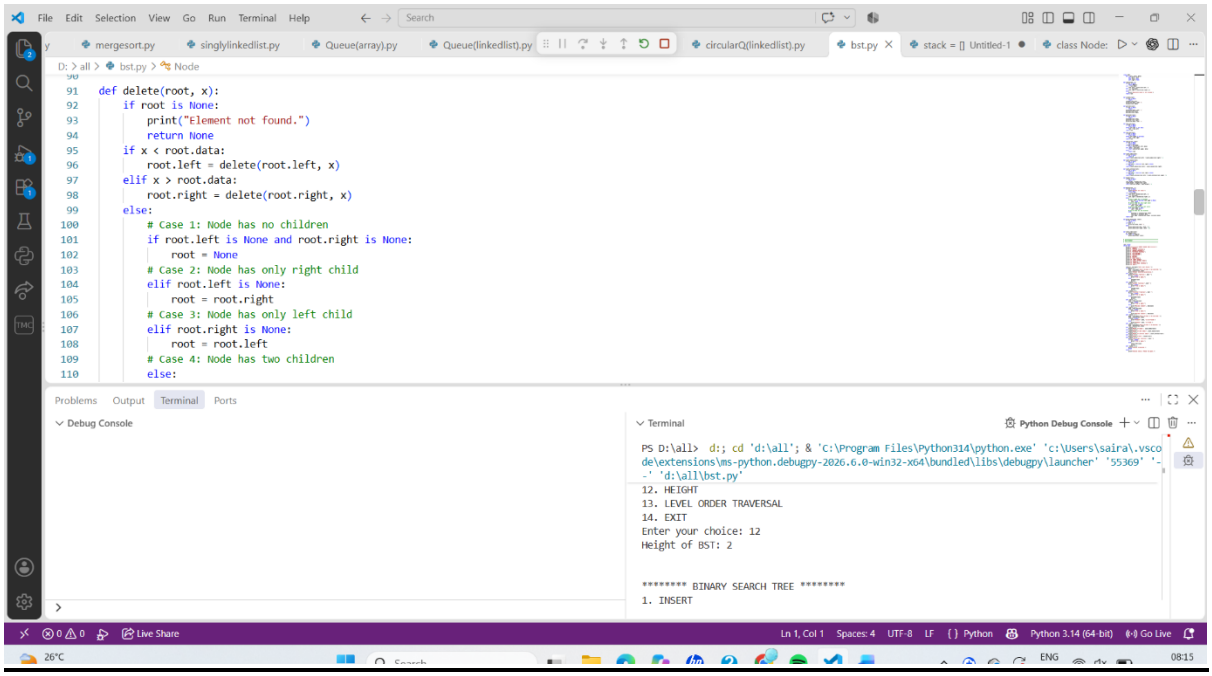
def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root


def inorder(root):
    if root is None:
        return
```

```
PS D:\all>  d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55369' '--' 'd:\all\bst.py'
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 2
Preorder Traversal: 23 34 56
```


File Edit Selection View Go Run Terminal Help
mergesort.py
singlylinkedlist.py
Queue(array).py
Queue(linkedlist).py
circularQ(linkedlist).py
bst.py
stack = [] Untitled-1
class Node:
def __init__(self, data):
self.data = data
self.left = None
self.right = None
def insert(root, x):
if root is None:
return Node(x)
if x < root.data:
root.left = insert(root.left, x)
elif x > root.data:
root.right = insert(root.right, x)
else:
print("Duplicate value is not allowed.")
return root
def inorder(root):
if root is None:
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vsco
de\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55369' '-
-' 'd:\all\bst.py'
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 10
Number of leaf nodes: 1


File Edit Selection View Go Run Terminal Help
mergesort.py
singlylinkedlist.py
Queue(array).py
Queue(linkedlist).py
circularQ(linkedlist).py
bst.py
stack = [] Untitled-1
class Node:
def __init__(self, data):
self.data = data
self.left = None
self.right = None
def insert(root, x):
if root is None:
return Node(x)
if x < root.data:
root.left = insert(root.left, x)
elif x > root.data:
root.right = insert(root.right, x)
else:
print("Duplicate value is not allowed.")
return root
def inorder(root):
if root is None:
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vsco
de\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55369' '-
-' 'd:\all\bst.py'
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 7
Enter the data to be searched: 23
Element 23 is found.
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14 (64-bit)
Go Live


def delete(root, x):
if root is None:
print("Element not found.")
return None
if x < root.data:
root.left = delete(root.left, x)
elif x > root.data:
root.right = delete(root.right, x)
else:
Case 1: Node has no children
if root.left is None and root.right is None:
root = None
Case 2: Node has only right child
elif root.left is None:
root = root.right
Case 3: Node has only left child
elif root.right is None:
root = root.left
Case 4: Node has two children
else:
Problems
Output
Terminal
Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vsco
de\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55369' '-
-' 'd:\all\bst.py'
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 12
Height of BST: 2
******** BINARY SEARCH TREE ********
1. INSERT

PROGRAM-10:

AIM: Write a menu driven python program to implement an AVL Tree and perform the following operations.

1. Insertion
2. Deletion
3. Tree Traversals
4. Count Leaf Nodes
5. Count Internal Nodes
6. Height of the Tree

CODE:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0


def height(root):
    if root is None:
        return -1
    else:
        return root.h
```

```
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

def RR(x):
    y = x.right
    x.right = y.left
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)

def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)

def inorder(root):
    if root is not None:
```

```
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
```

```
            root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
    root.h = max(height(root.left),height(root.right)) + 1
    return root

def delete(root, x):

    if root is None:
        return None

    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)
```

```
elif x > root.data:
    root.right = delete(root.right, x)
    if height(root.right) - height(root.left) == 2:
        if x > root.right.data:
            root = RR(root)
        else:
            root = RL(root)

else:
    if root.left is None and root.right is None:
        root = None
    elif root.right is None:
        root = root.left
    elif root.left is None:
        root = root.right
    else:
        successor = root.right
        while successor.left is not None:
            successor = successor.left
        root.data = successor.data
        root.right = delete(root.right, successor.data)

    root.h = max(height(root.left), height(root.right)) + 1
return root
```

```
root = None

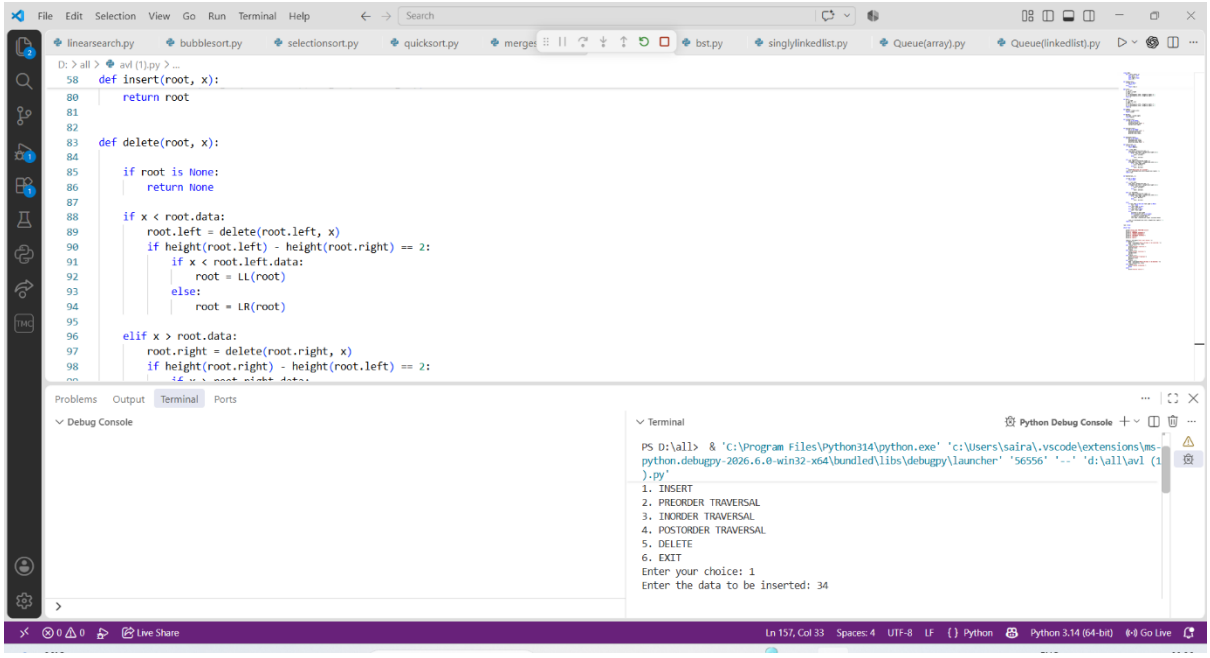
while True:

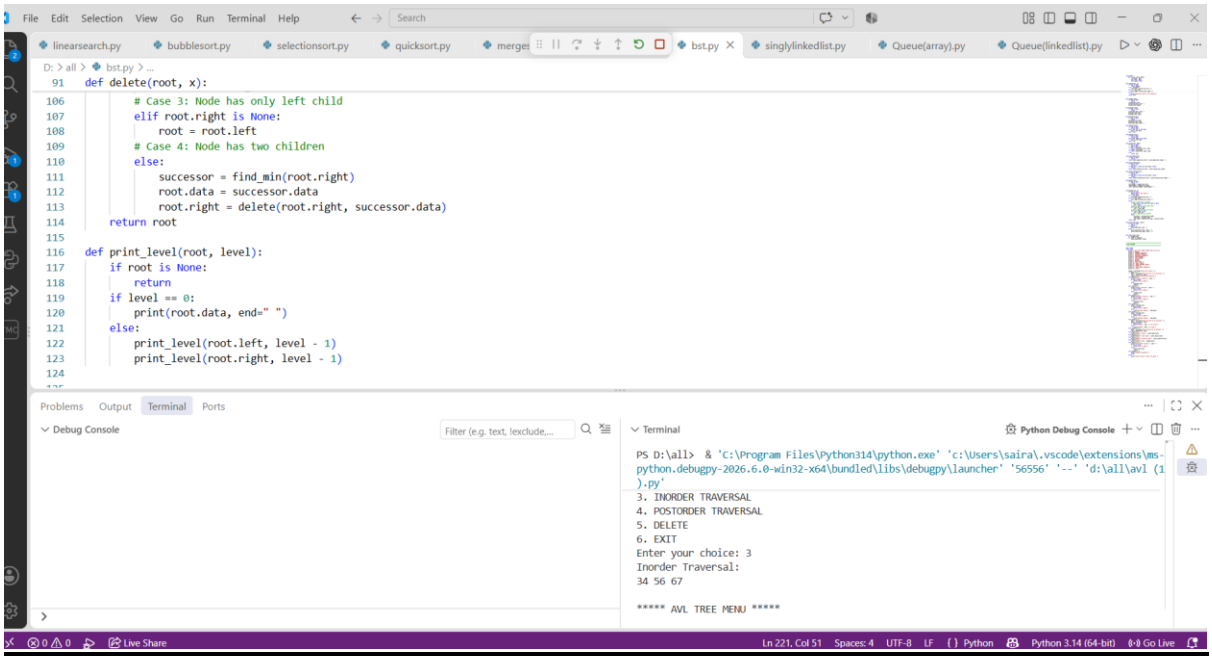
    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
    print("6. EXIT")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter the data to be inserted: "))
        root = insert(root, data)
    elif choice == 2:
        print("Preorder Traversal:")
        preorder(root)
        print()
    elif choice == 3:
        print("Inorder Traversal:")
        inorder(root)
        print()
    elif choice == 4:
        print("Postorder Traversal:")
```

```
        postorder(root)
        print()
    elif choice == 5:
        data = int(input("Enter the data to be deleted: "))
        root = delete(root, data)
    elif choice == 6:
        print("Program terminated.")
        break
    else:
        print("Invalid choice!")
```

OUTPUT:



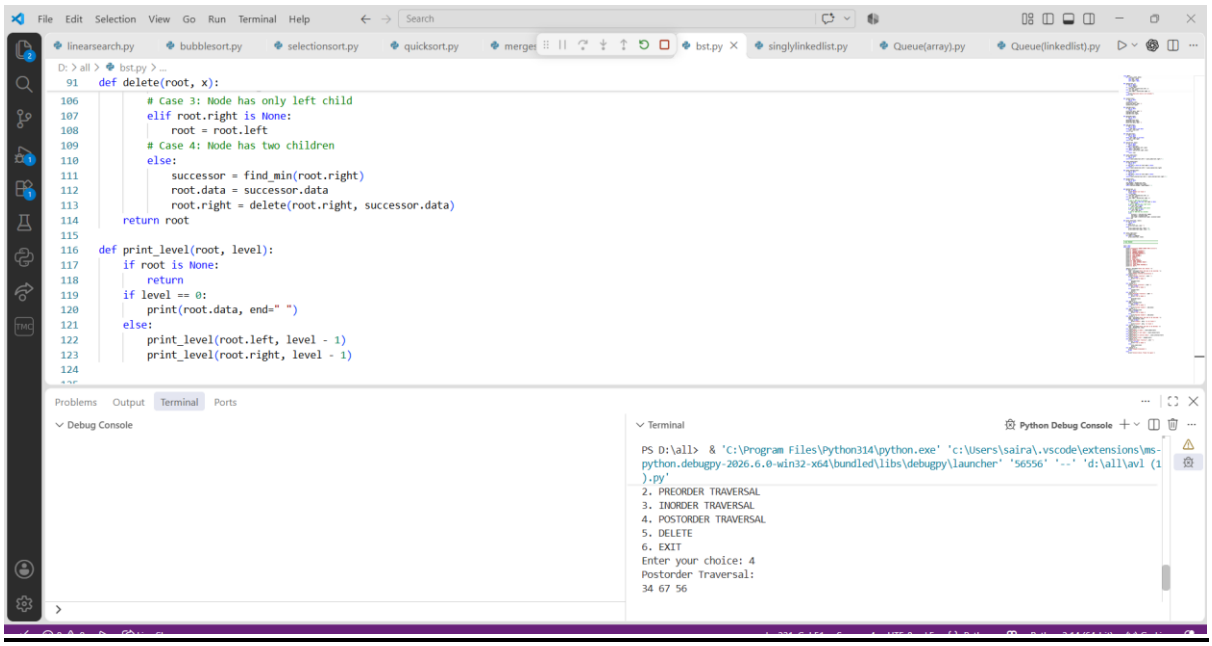


```
def delete(root, x):
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)
```

```
PS D:\all>  & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56556' '--' 'd:\all\avl (1).py'
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 3
Inorder Traversal:
34 56 67

***** AVL TREE MENU *****
```



```
def delete(root, x):
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)
```

```
PS D:\all>  & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56556' '--' 'd:\all\avl (1).py'
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 4
Postorder Traversal:
34 67 56
```

PROGRAM-11:

AIM: Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:

1. Insertion of an element into the Heap.
2. Deletion of the root element from the Heap.
3. Display the array representation of the Heap.
4. Exit the program.

CODE:

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
```

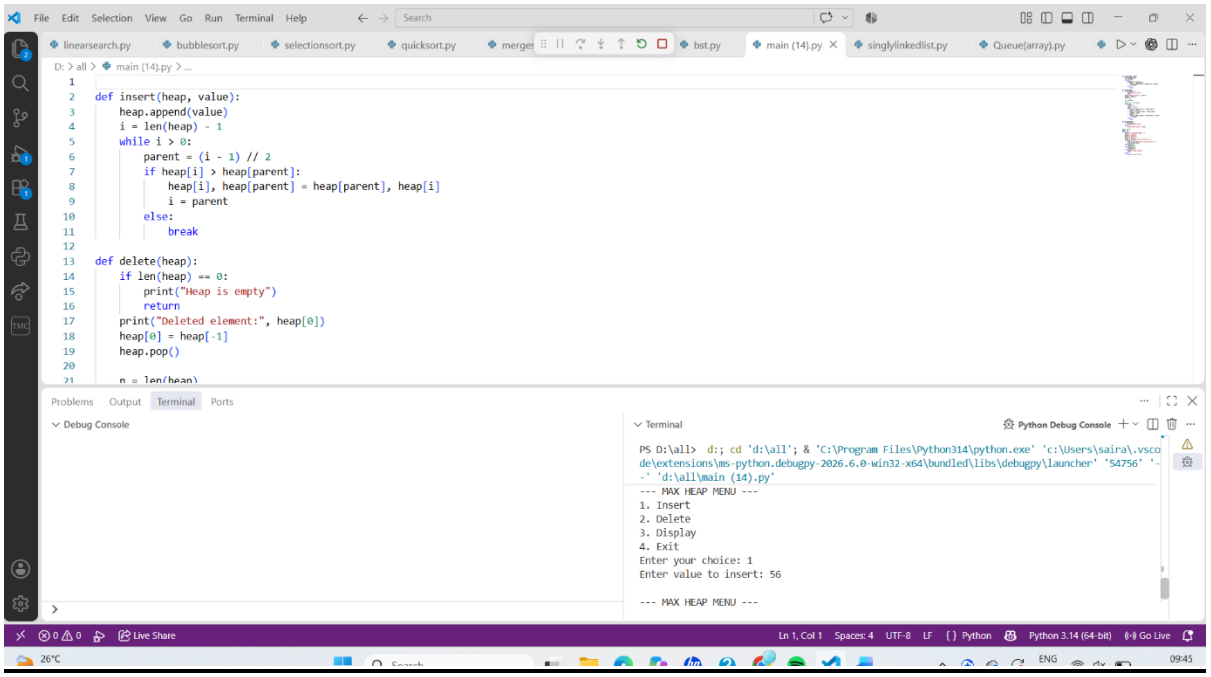
```
    heap[0] = heap[-1]
    heap.pop()

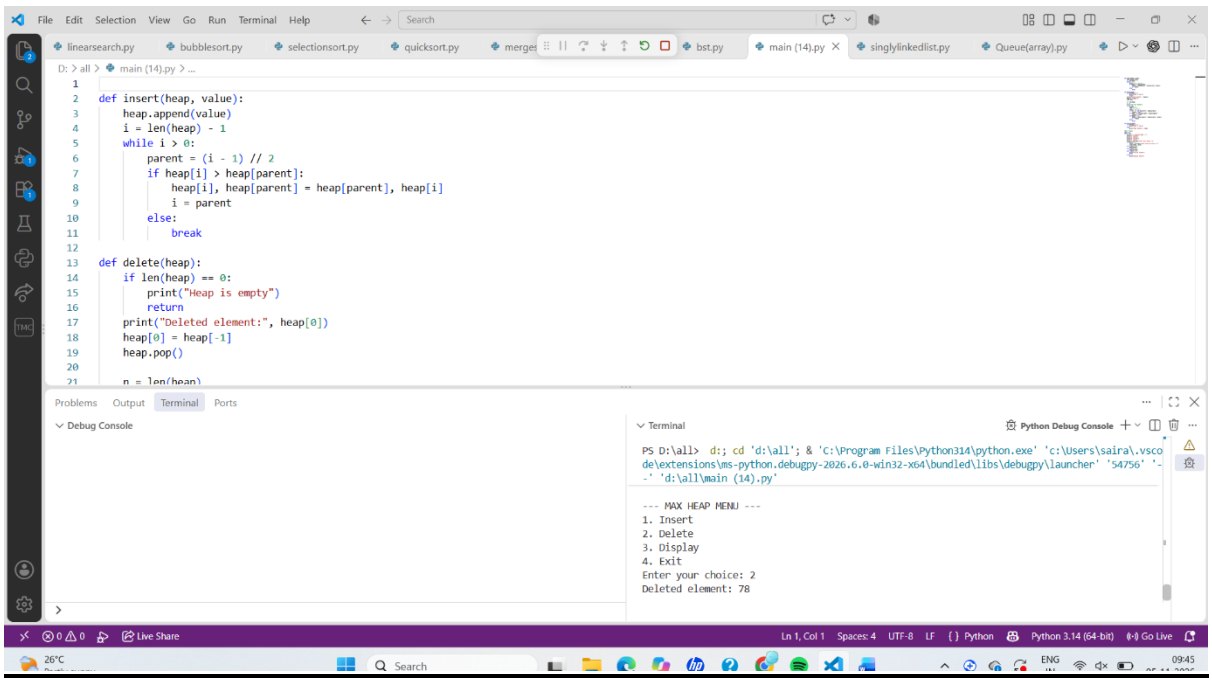
    n = len(heap)
    i = 0
    # Move the root downward
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest != i:
            heap[i], heap[largest] = heap[largest], heap[i]
            i = largest
        else:
            break

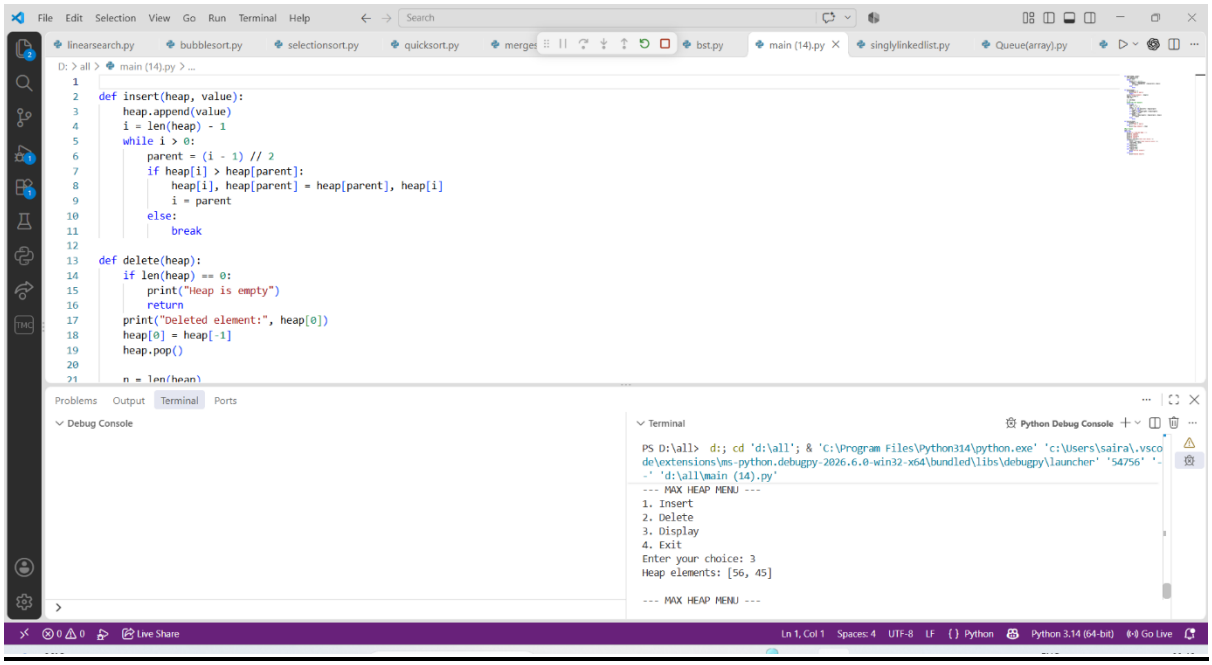
def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)
```

```
#Main Program
heap = []
while True:
    print("\n--- MAX HEAP MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)
    elif choice == 2:
        delete(heap)
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

OUTPUT:


def insert(heap, value):
heap.append(value)
i = len(heap) - 1
while i > 0:
parent = (i - 1) // 2
if heap[i] > heap[parent]:
heap[i], heap[parent] = heap[parent], heap[i]
i = parent
else:
break
def delete(heap):
if len(heap) == 0:
print("Heap is empty")
return
print("Deleted element:", heap[0])
heap[0] = heap[-1]
heap.pop()
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value to insert: 56
--- MAX HEAP MENU ---


def insert(heap, value):
heap.append(value)
i = len(heap) - 1
while i > 0:
parent = (i - 1) // 2
if heap[i] > heap[parent]:
heap[i], heap[parent] = heap[parent], heap[i]
i = parent
else:
break
def delete(heap):
if len(heap) == 0:
print("Heap is empty")
return
print("Deleted element:", heap[0])
heap[0] = heap[-1]
heap.pop()
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 2
Deleted element: 78


File Edit Selection View Go Run Terminal Help
Search
linearsearch.py
bubblesort.py
selectionsort.py
quicksort.py
bst.py
main (14).py
singlylinkedlist.py
Queue(array).py
D: > all > main (14).py > ...
def insert(heap, value):
heap.append(value)
i = len(heap) - 1
while i > 0:
parent = (i - 1) // 2
if heap[i] > heap[parent]:
heap[i], heap[parent] = heap[parent], heap[i]
i = parent
else:
break
def delete(heap):
if len(heap) == 0:
print("Heap is empty")
return
print("Deleted element:", heap[0])
heap[0] = heap[-1]
heap.pop()
Problems
Output
Terminal
Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vsco
de\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54756' '-
-' 'd:\all\main (14).py'
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Heap elements: [56, 45]
--- MAX HEAP MENU ---
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14 (64-bit)
Go Live

PROGRAM-12:

AIM: Write a program to implement the following three collision resolution techniques in a Hash Table:

1. Separate Chaining
2. Linear Probing
3. Quadratic Probing

CODE:

Separate Chaining

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

SIZE = 10
ht = [None] * SIZE

def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
```

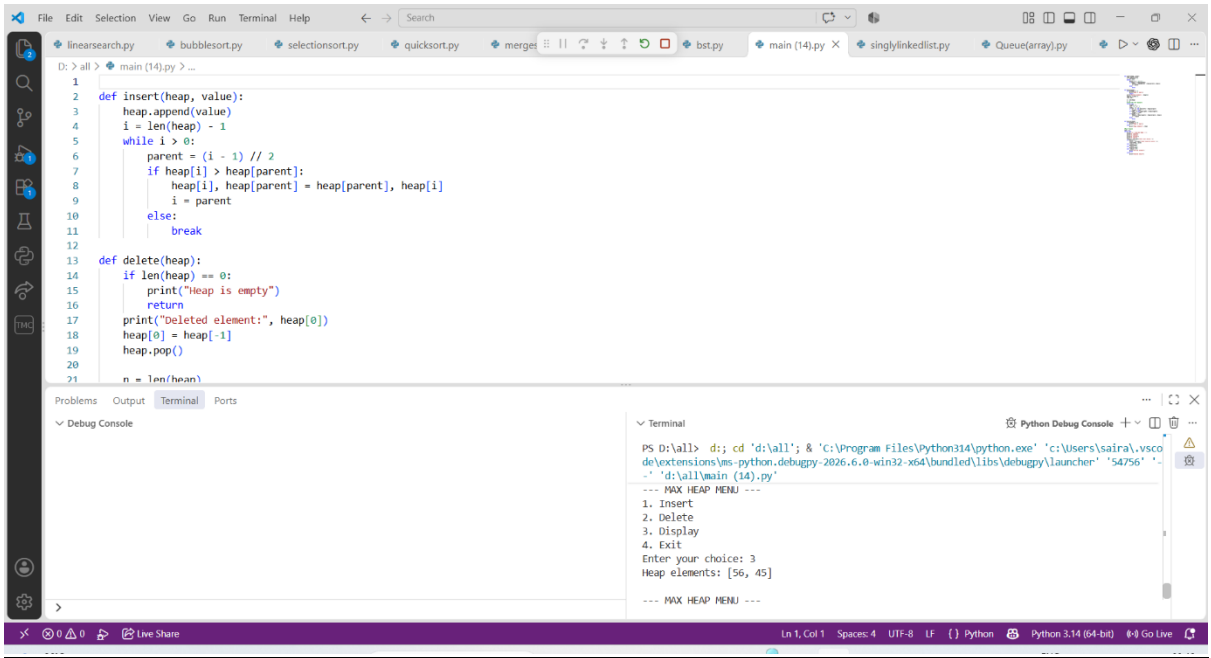
```
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode

def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
```

```
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:



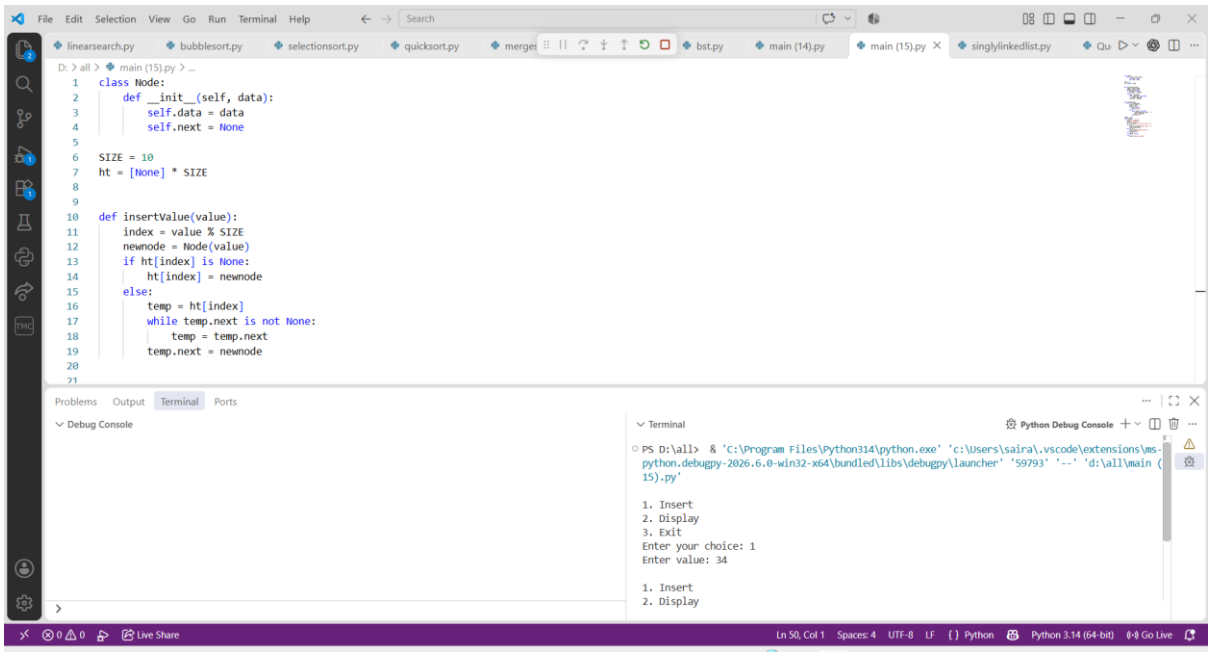
```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
```

```
PS D:\all>  d:; cd 'd:\all'; & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vsco
de\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54756' '-
-' 'd:\all\main (14).py'
--- MAX HEAP MENU ---
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Heap elements: [56, 45]

--- MAX HEAP MENU ---
```



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

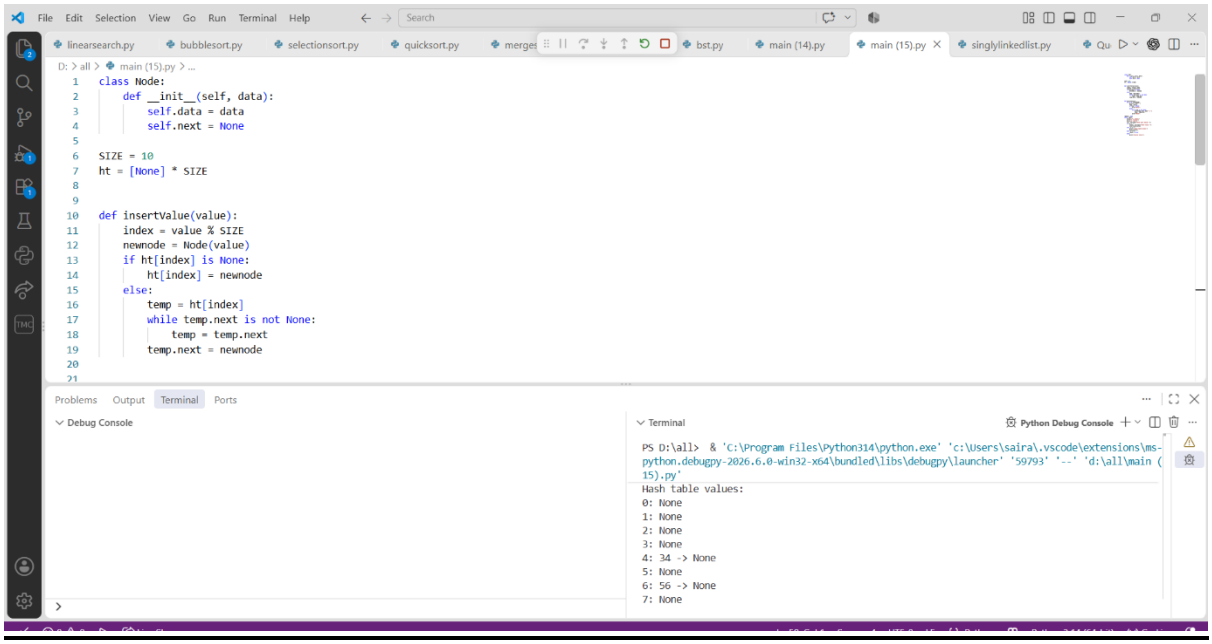
SIZE = 10
ht = [None] * SIZE


def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode
```

```
PS D:\all>  & 'c:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-
python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59793' '--' 'd:\all\main (
15).py'

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 34

1. Insert
2. Display
```


File Edit Selection View Go Run Terminal Help
Search
linearsearch.py
bubblesort.py
selectionsort.py
quicksort.py
bst.py
main (14).py
main (15).py
singlylinkedlist.py
D: > all > main (15).py > ...
class Node:
def __init__(self, data):
self.data = data
self.next = None
SIZE = 10
ht = [None] * SIZE
def insertValue(value):
index = value % SIZE
newnode = Node(value)
if ht[index] is None:
ht[index] = newnode
else:
temp = ht[index]
while temp.next is not None:
temp = temp.next
temp.next = newnode
Problems
Output
Terminal
Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '59793' '--' 'd:\all\main (15).py'
Hash table values:
0: None
1: None
2: None
3: None
4: 34 -> None
5: None
6: 56 -> None
7: None

Linear probing

CODE:

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")

def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
```

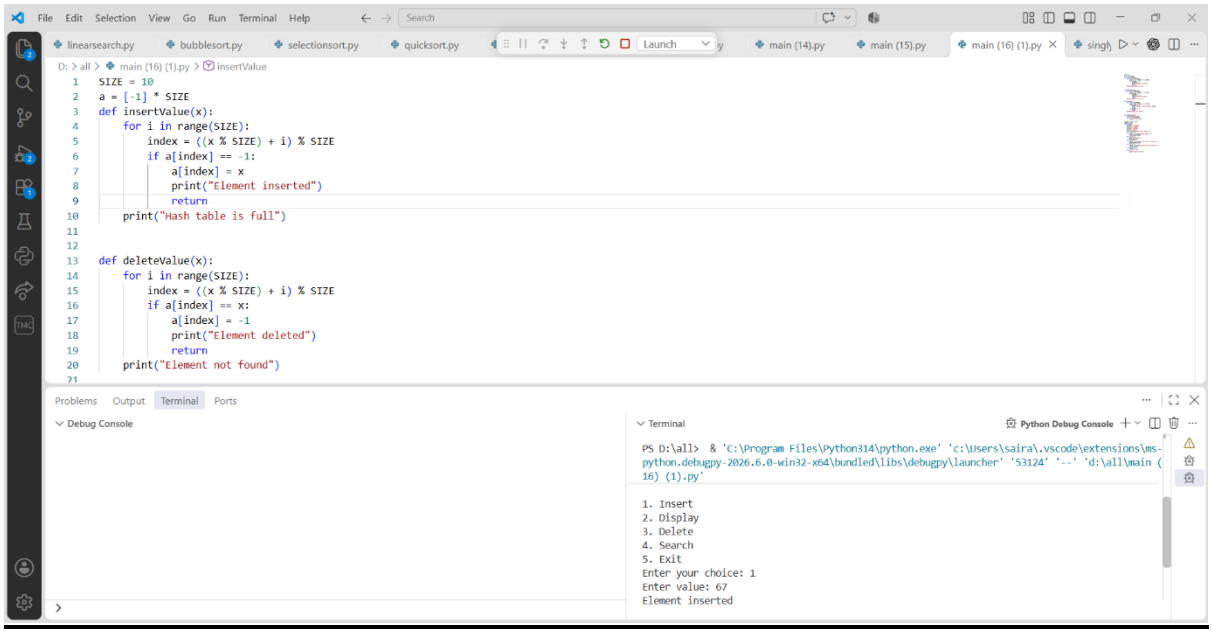
```
            print("Element found at index", index)
            return
        if a[index] == -1:
            break
    print("Element not found")

def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
```

```
    elif ch == 3:
        value = int(input("Enter value to delete: "))
        deleteValue(value)
    elif ch == 4:
        value = int(input("Enter value to search: "))
        search(value)
    elif ch == 5:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:

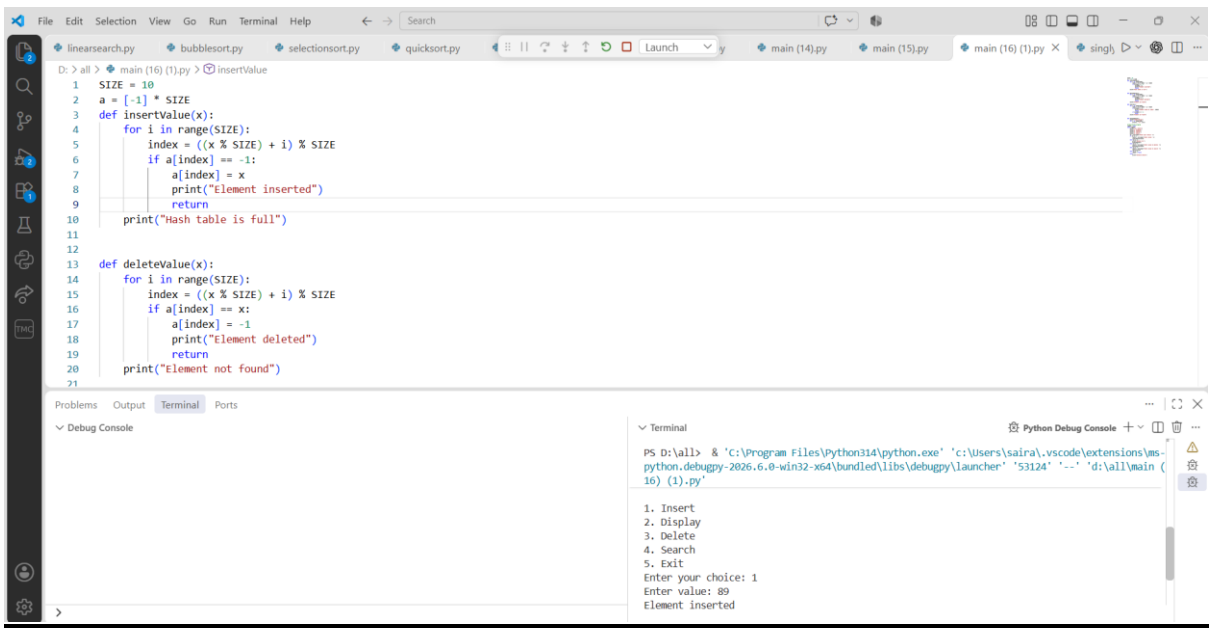


```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53124' '--' 'd:\all\main (16) (1).py'

1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 67
Element inserted
```

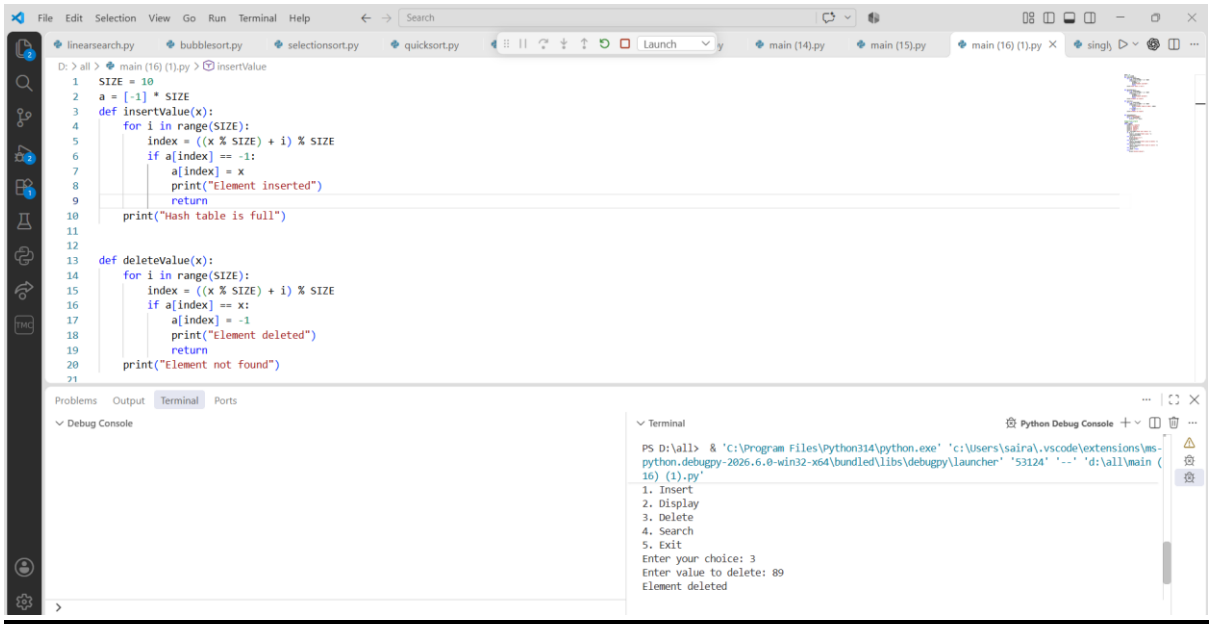


```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53124' '--' 'd:\all\main (16) (1).py'

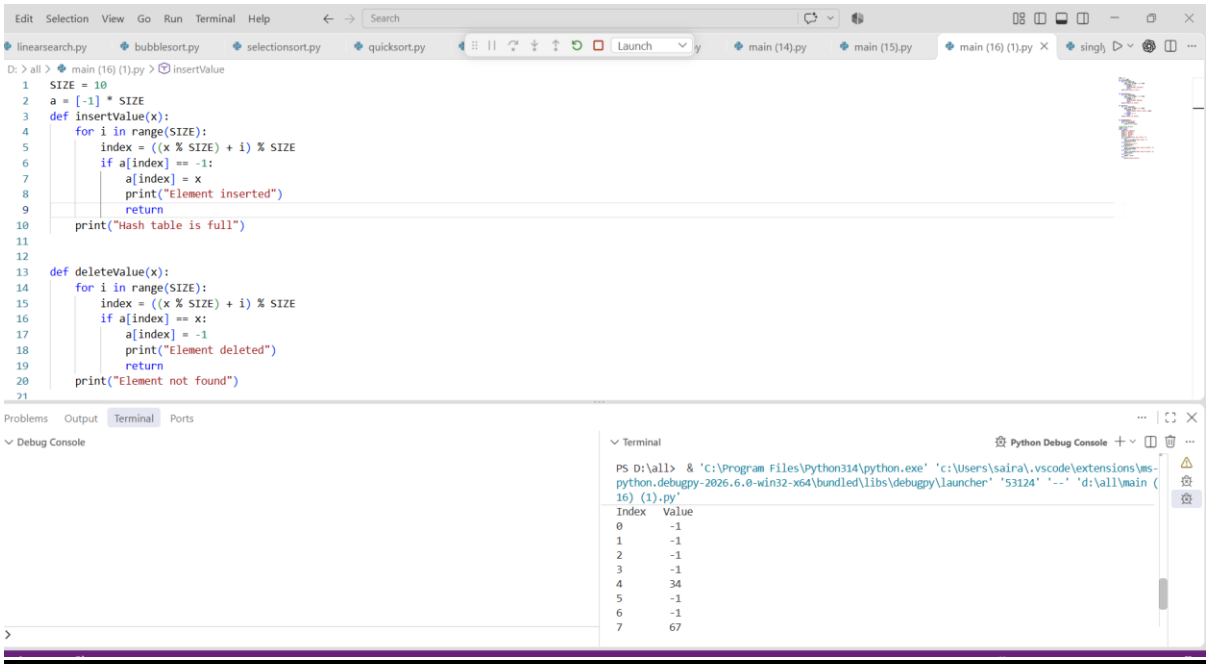
1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 1
Enter value: 89
Element inserted
```



```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53124' '--' 'd:\all\main (16) (1).py'
1. Insert
2. Display
3. Delete
4. Search
5. Exit
Enter your choice: 3
Enter value to delete: 89
Element deleted
```



```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")


def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")
```

```
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53124' '--' 'd:\all\main (16) (1).py'
Index   Value
0       -1
1       -1
2       -1
3       -1
4       34
5       -1
6       -1
7       67
```

Quadratic probing

CODE:

```
SIZE = 10
a = [-1] * SIZE

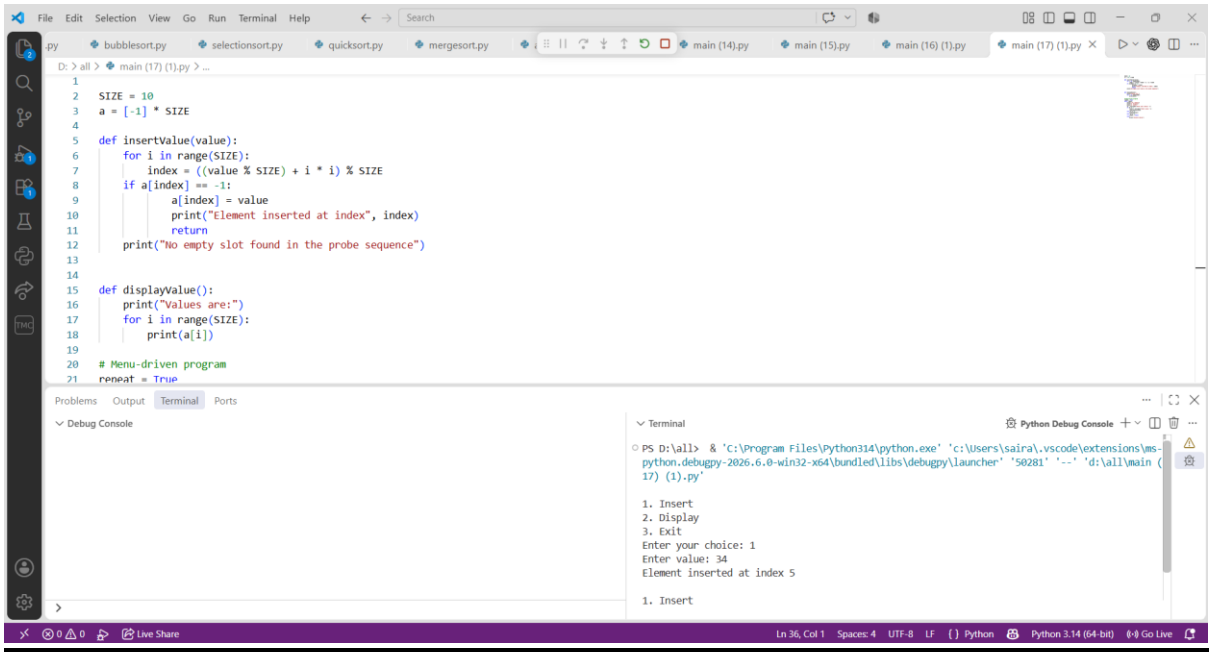
def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
    if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")

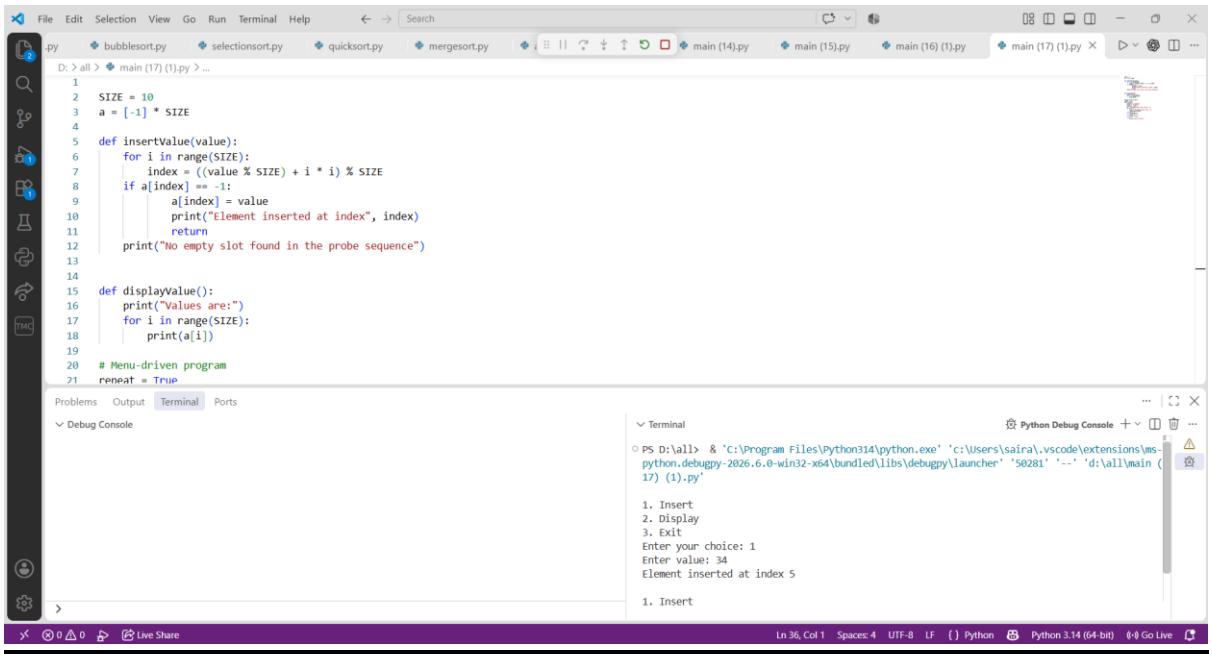
def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
```

```
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

OUTPUT:


File Edit Selection View Go Run Terminal Help
bubblesort.py
selectionsort.py
quicksort.py
mergesort.py
main (14).py
main (15).py
main (16) (1).py
main (17) (1).py
SIZE = 10
a = [-1] * SIZE
def insertValue(value):
for i in range(SIZE):
index = ((value % SIZE) + i * i) % SIZE
if a[index] == -1:
a[index] = value
print("Element inserted at index", index)
return
print("No empty slot found in the probe sequence")
def displayValue():
print("Values are:")
for i in range(SIZE):
print(a[i])
Menu-driven program
repeat = True
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50281' '--' 'd:\all\main (17) (1).py'
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 34
Element inserted at index 5
1. Insert
Live Share
Ln 36, Col 1 Spaces: 4 UTF-8 LF Python Python 3.14 (64-bit) Go Live


File Edit Selection View Go Run Terminal Help
bubblesort.py
selectionsort.py
quicksort.py
mergesort.py
main (14).py
main (15).py
main (16) (1).py
main (17) (1).py
SIZE = 10
a = [-1] * SIZE
def insertValue(value):
for i in range(SIZE):
index = ((value % SIZE) + i * i) % SIZE
if a[index] == -1:
a[index] = value
print("Element inserted at index", index)
return
print("No empty slot found in the probe sequence")
def displayValue():
print("Values are:")
for i in range(SIZE):
print(a[i])
Menu-driven program
repeat = True
Problems Output Terminal Ports
Debug Console
Terminal
Python Debug Console
PS D:\all> & 'C:\Program Files\Python314\python.exe' 'c:\Users\saira\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '50281' '--' 'd:\all\main (17) (1).py'
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 34
Element inserted at index 5
1. Insert
Live Share
Ln 36, Col 1 Spaces: 4 UTF-8 LF Python Python 3.14 (64-bit) Go Live